

Laporan Akhir Tugas Besar Pemrograman Berorientasi bjek CINEMA TICKET



IRZI ARINTA DWI PONILAN	103012400028
TIANISA SANIPAR	103012400213
SHAKIRA BILQIS SARWAHITA	103012400266
SITI ALQIA TONGGIROH	103012300331
NADINE NAFEESA SETYAWAN	103012400133
NAILA AMALIA	103012400295

**S1 Informatika
Fakultas Informatika
Universitas Telkom
2026**

Daftar Isi

Laporan Akhir Tugas Besar	1
Pemrograman Berorientasi objek.....	1
Daftar Isi.....	1
Daftar Table	4
Daftar Gambar.....	6
1 PENDAHULUAN	9
1.1 Latar Belakang.....	9
1.2 Tujuan.....	9
1.3 Deskripsi Umum Sistem (Ruang Lingkup)	10
1.4 Metodologi Pengembangan Sistem	10
2 DASAR TEORI	11
2.1 Pemrograman Berorientasi Objek	11
2.2 Unified Modeling Language (UML)	11
2.3 Java.....	12
2.4 PostgreSQL/Supabase	12
2.5 Spring Boot dan Flutter.....	12
2.6 Entity Relationship Diagram (ERD)	12
3 ANALISIS DAN IMPLEMENTASI SISTEM.....	12
3.1 ANALISIS KEBUTUHAN SISTEM	12
3.2 IMPLEMENTASI BASIS DATA.....	15
3.3 IMPLEMENTASI UML.....	18

3.4	IMPLEMENTASI ANTARMUKA PENGGUNA (USER INTERFACE).....	23
4	PENGUJIAN SISTEM	23
4.1	Tujuan Pengujian.....	23
4.2	Lingkungan Pengujian.....	24
4.3	Metode Pengujian	24
5	HASIL UJI	25
5.1	Pengujian Autentikasi dan Manajemen Akun	26
5.2	Pengujian Layanan Proses Booking.....	33
5.3	Pengujian Kelas Booking.....	41
5.4	Pengujian Kelas Customer	45
5.5	Pengujian Kelas Admin.....	49
5.6	Pengujian Kelas Movie	54
5.7	Pengujian Kelas Schedule.....	56
5.8	Pengujian Kelas Seat	58
5.9	Pengujian Kelas Ticket.....	60
5.10	Pengujian Kelas Transaction	62
5.11	Pengujian Kelas VirtualAccountPayment	64
5.12	Bukti Running Testing	65
6	PENUTUP	66
7	DAFTAR PUSTAKA.....	67
8	LAMPIRAN	67
8.1	Lampiran A Pembagian Tugas	67

8.2	Lampiran B Source Code Penting.....	67
8.3	Lampiran C Screenshot Sistem.....	68
8.4	Lampiran D Manual Instalasi Sistem.....	81

Daftar Table

1	Tabel Entitas User	19
2	Identifikasi Pengujian	25
3	Kasus Uji Pengujian Autentikasi dan Manajemen Akun	32
4	Hasil Uji Pengujian Autentikasi dan Manajemen Akun	32
5	Kasus Uji Pengujian Layanan Proses Booking	40
6	Hasil Uji Pengujian Layanan Proses Booking	41
7	Kasus Uji Pengujian Kelas Booking	44
8	Hasil Uji Pengujian Kelas Booking	45
9	Kasus Uji Pengujian Kelas Customer	48
10	Hasil Uji Pengujian Kelas Customer	49
11	Kasus Uji Pengujian Kelas Admin	53
12	Hasil Uji Pengujian Kelas Admin	53
13	Kasus Uji Pengujian Kelas Movie	56
14	Hasil Uji Pengujian Kelas Movie	56
15	Kasus Uji Pengujian Kelas Schedule	57
16	Hasil Uji Pengujian Kelas Schedule	58
17	Kasus Uji Pengujian Kelas Seat	60
18	Hasil Uji Pengujian Kelas Seat	60
19	Kasus Uji Pengujian Kelas Ticket	62
20	Hasil Uji Pengujian Kelas Ticket	62
21	Kasus Uji Pengujian Kelas Transaction	64
22	Hasil Uji Pengujian Kelas Transaction	64
23	Kasus Uji Pengujian Kelas VirtualAccountPayment	66

24	Hasil Uji Pengujian Kelas VirtualAccountPayment.....	66
25	Pembagian Tugas Anggota Kelompok	68

Daftar Gambar

1	Entity Relationship Diagram terbaru sistem <i>Cinema Ticket</i>	18
2	Class Diagram terbaru sistem <i>Cinema Ticket</i>	21
3	Dokumentasi kode uji kelas AuthService bagian 1	27
4	Dokumentasi kode uji kelas AuthService bagian 2	28
5	Dokumentasi kode uji kelas AuthService bagian 3	29
6	Dokumentasi kode uji kelas AuthService bagian 4	30
7	Dokumentasi kode uji kelas AuthService bagian 5	31
8	Dokumentasi kode uji kelas AuthService bagian 6	31
9	Dokumentasi kode uji layanan BookingService bagian 1	34
10	Dokumentasi kode uji layanan BookingService bagian 2.....	35
11	Dokumentasi kode uji layanan BookingService bagian 3	36
12	Dokumentasi kode uji layanan BookingService bagian 4.....	37
13	Dokumentasi kode uji layanan BookingService bagian 5.....	38
14	Dokumentasi kode uji layanan BookingService bagian 6.....	39
15	Dokumentasi kode uji layanan BookingService bagian 7.....	40
16	Dokumentasi kode uji kelas Booking bagian 1	42
17	Dokumentasi kode uji kelas Booking bagian 2	43
18	Dokumentasi kode uji kelas Booking bagian 3	44
19	Dokumentasi kode uji kelas Customer bagian 1.....	46
20	Dokumentasi kode uji kelas Customer bagian 2.....	47
21	Dokumentasi kode uji kelas Customer bagian 3.....	48
22	Dokumentasi kode uji kelas Admin bagian 1.....	50
23	Dokumentasi kode uji kelas Admin bagian 2.....	51

24	Dokumentasi kode uji kelas Admin bagian 3.....	52
25	Dokumentasi kode uji kelas Admin bagian 4.....	53
26	Dokumentasi kode uji kelas Movie bagian 1.....	55
27	Dokumentasi kode uji kelas Schedule bagian 1	57
28	Dokumentasi kode uji kelas Seat bagian 1.....	59
29	Dokumentasi kode uji kelas Ticket bagian 1	61
30	Dokumentasi kode uji kelas Transaction bagian 1	63
31	Dokumentasi kode uji kelas VirtualAccountPayment bagian 1.....	65
32	Bukti running testing bagian 1.....	67
33	Bukti running testing bagian 2.....	67
34	Halaman login user.....	69
35	Dashboard customer	70
36	Halaman pencarian film.....	71
37	Halaman pemilihan kursi customer	72
38	Halaman pembayaran customer	73
39	Halaman pembayaran QRIS.....	74
40	Invoice pembayaran berhasil.....	75
41	Daftar tiket customer.....	76
42	Halaman profil customer.....	77
43	Halaman profil admin.....	78
44	Halaman manajemen movie admin	79
45	Halaman manajemen schedule admin	80
46	Halaman manajemen iklan	81
47	Konfigurasi dan status koneksi Supabase	82

48	Backend Spring Boot berjalan.....	83
49	Frontend Flutter berjalan	83

1 PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi mendorong transformasi proses layanan publik dan komersial ke dalam bentuk digital, termasuk layanan pemesanan tiket bioskop. Sistem pemesanan tiket secara konvensional sering menimbulkan kendala seperti antrean panjang, kesalahan pencatatan jadwal, keterbatasan informasi kursi yang tersedia, serta proses transaksi yang kurang efisien. Oleh karena itu, diperlukan sebuah sistem pemesanan tiket bioskop yang mampu mengelola data pengguna, film, jadwal, kursi, booking, transaksi, tiket, dan iklan secara terstruktur.

Aplikasi *Cinema Ticket* dirancang sebagai sistem pemesanan tiket bioskop berbasis konsep Pemrograman Berorientasi Objek. Diagram UML Class Diagram menunjukkan bahwa sistem dibangun melalui kelas-kelas utama seperti *User*, *Customer*, *Admin*, *Movie*, *Schedule*, *Seat*, *Transaction*, *Ticket*, *AuthService*, *VirtualAccountPayment*, *Advertisement*, dan interface *Payment*. Setiap kelas memiliki tanggung jawab yang berbeda sehingga sistem menjadi modular, mudah dikembangkan, dan sesuai dengan prinsip OOP.

Dari sisi penyimpanan data, Entity Relationship Diagram menunjukkan adanya entitas *users*, *movies*, *schedules*, *seats*, *bookings*, *booking seats*, *tickets*, *transactions*, dan *advertisements*. Relasi antartabel tersebut menggambarkan alur data mulai dari pengguna melakukan pemesanan jadwal film, memilih kursi, membentuk data booking, menghasilkan tiket, hingga mencatat transaksi pembayaran.

1.2 Tujuan

Adapun tujuan dari penyusunan dan pengembangan sistem *Cinema Ticket* adalah sebagai berikut:

1. Mengimplementasikan konsep Pemrograman Berorientasi Objek yang meliputi inheritance, encapsulation, polymorphism, dan abstraction.
2. Merancang struktur kelas sistem pemesanan tiket bioskop berdasarkan UML Class Diagram.
3. Merancang struktur basis data berdasarkan Entity Relationship Diagram yang mencakup entitas, atribut, primary key, foreign key, dan relasi data.
4. Menjelaskan alur kerja sistem mulai dari autentikasi pengguna, pengelolaan film, penjadwalan, pemilihan kursi, booking, pembayaran, sampai penerbitan tiket.
5. Menghasilkan laporan akademik yang sesuai dengan format template laporan tugas besar.

1.3 Deskripsi Umum Sistem (Ruang Lingkup)

Cinema Ticket merupakan aplikasi pemesanan tiket bioskop yang menyediakan layanan bagi dua jenis pengguna utama, yaitu *Customer* dan *Admin*. *Customer* dapat melakukan registrasi, login, melihat daftar film, memilih jadwal tayang, memilih kursi, melakukan booking, melakukan pembayaran, melihat tiket, dan melihat riwayat booking. *Admin* memiliki kewenangan untuk mengelola data film, jadwal tayang, kursi, dan iklan.

Ruang lingkup sistem meliputi:

1. Manajemen akun pengguna melalui kelas *User*, *Customer*, *Admin*, dan *AuthService*.
2. Manajemen data film melalui kelas *Movie* dan tabel *movies*.
3. Manajemen jadwal tayang melalui kelas *Schedule* dan tabel *schedules*.
4. Manajemen ketersediaan kursi melalui kelas *Seat*, tabel *seats*, dan tabel penghubung *booking-seats*.
5. Proses booking melalui data *Booking* pada atribut dan tabel *bookings*.
6. Proses pembayaran melalui interface *Payment* dan implementasi *VirtualAccountPayment*.
7. Pencatatan transaksi pembayaran melalui kelas *Transaction* dan tabel *transactions*, dengan relasi satu-ke-satu terhadap *Booking*.
8. Penerbitan tiket melalui kelas *Ticket* dan tabel *tickets*.
9. Manajemen iklan/promosi melalui kelas dan tabel *Advertisement*.

1.4 Metodologi Pengembangan Sistem

Metodologi pengembangan sistem dilakukan melalui tahapan analisis kebutuhan, perancangan model sistem, implementasi kelas dan basis data, serta pengujian fitur utama. Diagram UML digunakan sebagai acuan perancangan objek, sedangkan ERD digunakan sebagai acuan penyusunan struktur penyimpanan data.

1.4.1 Analisis Kebutuhan

Analisis kebutuhan dilakukan dengan mengidentifikasi aktor sistem, fitur utama, serta data yang dibutuhkan. Aktor utama terdiri atas *Customer* dan *Admin*. Kebutuhan fungsional utama mencakup autentikasi, pengelolaan film, pengelolaan jadwal, pemilihan kursi, pembuatan booking, pembayaran, penerbitan tiket, dan pengelolaan iklan.

1.4.2 Perancangan Sistem

Perancangan sistem dilakukan menggunakan UML Class Diagram untuk menentukan struktur kelas, atribut, method, inheritance, interface, serta asosiasi antarkelas. ERD digunakan untuk menentukan tabel, atribut, primary key, foreign key, serta relasi antarentitas.

1.4.3 Implementasi Sistem

Implementasi sistem dilakukan dengan menerapkan struktur kelas dan basis data sesuai rancangan. Setiap kelas memiliki atribut yang dienkapsulasi dan method yang merepresentasikan perilaku objek. Basis data digunakan untuk menyimpan data pengguna, film, jadwal, kursi, booking, tiket, transaksi, dan iklan.

2 DASAR TEORI

2.1 Pemrograman Berorientasi Objek

Pemrograman Berorientasi Objek atau Object-Oriented Programming (OOP) merupakan paradigma pemrograman yang memodelkan sistem ke dalam objek. Objek memiliki atribut sebagai representasi data dan method sebagai representasi perilaku. Pada sistem *Cinema Ticket*, objek direpresentasikan oleh kelas seperti *User*, *Customer*, *Admin*, *Movie*, *Schedule*, *Seat*, *Transaction*, dan *Ticket*.

Konsep OOP yang diterapkan meliputi:

1. *Encapsulation*, yaitu pembatasan akses atribut dengan penggunaan atribut private dan penyediaan method publik untuk mengakses atau mengubah data.
2. *Inheritance*, yaitu pewarisan atribut dan perilaku dari kelas induk *User* kepada kelas turunan *Customer* dan *Admin*.
3. *Polymorphism*, yaitu kemampuan method dengan nama yang sama untuk memiliki perilaku berbeda pada kelas yang berbeda, misalnya method *getRole()* pada *Customer* dan *Admin*.
4. *Abstraction*, yaitu penyederhanaan proses pembayaran melalui interface *Payment* yang mendefinisikan operasi umum tanpa menampilkan detail implementasi.

2.2 Unified Modeling Language (UML)

Unified Modeling Language (UML) adalah bahasa pemodelan standar yang digunakan untuk menggambarkan struktur dan perilaku sistem perangkat lunak. Pada laporan ini, UML yang digunakan adalah Class Diagram. Class Diagram menggambarkan kelas, atribut, method, relasi inheritance, implementasi interface, asosiasi, agregasi, dan komposisi dalam sistem *Cinema Ticket*.

2.3 Java

Java merupakan bahasa pemrograman berorientasi objek yang mendukung penerapan class, object, inheritance, interface, encapsulation, dan polymorphism. Struktur pada UML Class Diagram dapat diimplementasikan dalam Java melalui deklarasi class, atribut private, method public, extends untuk inheritance, dan implements untuk interface.

2.4 PostgreSQL/Supabase

PostgreSQL merupakan sistem manajemen basis data relasional yang digunakan pada backend proyek *Cinema Ticket*. Konfigurasi backend juga mendukung Supabase melalui variabel lingkungan *DATABASE_URL*, *DB_USERNAME*, dan *DB_PASSWORD*. Model ERD pada laporan ini mengikuti struktur tabel yang dibentuk oleh Spring Data JPA dan Hibernate pada PostgreSQL/Supabase.

2.5 Spring Boot dan Flutter

Spring Boot merupakan framework Java yang digunakan pada backend proyek *Cinema Ticket* untuk menyediakan REST API. Flutter dan Dart digunakan pada sisi frontend/mobile untuk menampilkan daftar film, jadwal, kursi, booking, tiket, profil, pencarian, serta halaman admin. Komunikasi frontend dan backend dilakukan melalui endpoint REST seperti */api/auth*, */api/movies*, */api/schedules*, */api/seats*, */api/bookings*, */api/ads*, dan */api/uploads*. Backend berjalan pada port 8082 sesuai konfigurasi *application.properties* dan *app config.dart*.

2.6 Entity Relationship Diagram (ERD)

Entity Relationship Diagram adalah diagram yang menggambarkan struktur data dalam basis data relasional. ERD pada sistem *Cinema Ticket* menampilkan entitas *users*, *movies*, *schedules*, *seats*, *bookings*, *booking seats*, *tickets*, *transactions*, dan *advertisements* beserta atribut dan relasinya.

3 ANALISIS DAN IMPLEMENTASI SISTEM

3.1 ANALISIS KEBUTUHAN SISTEM

Analisis kebutuhan sistem disusun berdasarkan fitur yang muncul pada Class Diagram dan ERD. Setiap fitur memiliki keterkaitan langsung dengan kelas dan tabel tertentu sehingga rancangan sistem dapat ditelusuri dari sisi objek maupun penyimpanan data.

3.1.1 KEBUTUHAN FUNGSIONAL (FITUR)

Kebutuhan fungsional sistem meliputi fitur-fitur utama berikut.

3.1.1.1. FITUR AUTENTIKASI PENGGUNA

Fitur autentikasi dikelola oleh kelas *AuthService* yang memiliki atribut *users: List<User>* dan method *register()*, *login()*, *logout()*, serta *changePassword()*. Data autentikasi tersimpan pada tabel *users* dengan atribut *user id*, *user type*, *email*, *name*, dan *password*.

3.1.1.2. FITUR PENGELOLAAN FILM

Fitur pengelolaan film direpresentasikan oleh kelas *Movie* dengan atribut *id*, *title*, *code*, *genre*, *duration*, *synopsis*, *price*, *imageUrl*, dan *status*. Pada ERD, data film disimpan pada tabel *movies* dengan atribut *id*, *duration*, *genre*, *image_url*, *price*, *status*, *synopsis*, *title*, dan *code*.

3.1.1.3. FITUR PENGELOLAAN JADWAL

Fitur pengelolaan jadwal direpresentasikan oleh kelas *Schedule* dengan atribut *scheduleId*, *hall*, *movie*, *date*, *time*, dan *endTime*. Pada ERD, tabel *schedules* memiliki atribut *schedule_id*, *date*, *hall*, *time*, *movie_id*, dan *end time*. Atribut *movie_id* menjadi foreign key yang menghubungkan jadwal dengan film.

3.1.1.4. FITUR PEMILIHAN KURSI

Fitur pemilihan kursi direpresentasikan oleh kelas *Seat* dengan atribut *id*, *seatNumber*, *isBooked*, dan relasi *schedule*. Method *bookSeat()* mengubah status kursi menjadi terpesan, sedangkan *isAvailable()* memeriksa ketersediaan kursi. Pada ERD, data kursi disimpan pada tabel *seats* dan hubungan kursi dengan booking disimpan pada tabel *booking_seats*.

3.1.1.5. FITUR BOOKING TIKET

Fitur booking terdapat pada atribut *bookingHistory: List<Booking>* milik *Customer*, atribut *booking: Booking* milik *Transaction*, dan atribut *booking id* pada *Ticket*. Pada ERD, proses booking disimpan dalam tabel *bookings* yang memiliki atribut *id*, *booking code*, *status*, *total price*, *schedule_id*, dan *user id*.

3.1.1.6. FITUR PEMBAYARAN

Fitur pembayaran diabstraksikan melalui interface *Payment* yang memiliki method *pay()*, *validate()*, *generateInvoice()*, dan *getPaymentInfo()*. Implementasi konkretnya adalah kelas *VirtualAccountPayment* dengan atribut *walletType*, *phoneNumber*, dan *walletBalance*. Hal ini menunjukkan bahwa sistem dapat dikembangkan untuk metode pembayaran lain tanpa mengubah

kontrak utama pembayaran.

3.1.1.7. FITUR TRANSAKSI DAN TIKET

Fitur transaksi direpresentasikan oleh kelas *Transaction* dengan atribut *transactionId*, *booking*, *total*, *status*, dan *transactionDate*. Tiket direpresentasikan oleh kelas *Ticket* dengan atribut *ticketId*, *ticketCode*, serta relasi *booking* dan *seat*. Pada ERD, data transaksi tersimpan pada tabel *transactions*, sedangkan tiket tersimpan pada tabel *tickets*.

3.1.2 KEBUTUHAN NON-FUNGSIONAL

Kebutuhan non-fungsional sistem meliputi keamanan data, konsistensi data, kemudahan pemeliharaan, dan ketersediaan sistem. Keamanan diterapkan melalui penyimpanan atribut sensitif seperti *password* secara terbatas pada kelas *User*. Konsistensi data dijaga melalui relasi primary key dan foreign key pada ERD. Kemudahan pemeliharaan dicapai melalui struktur class yang modular dan penerapan prinsip OOP.

3.1.2.1. SPESIFIKASI PERANGKAT LUNAK

Spesifikasi perangkat lunak yang digunakan dalam pengembangan dan pengujian sistem adalah sebagai berikut:

- Sistem Operasi: Windows, Linux, atau macOS.
- Bahasa Pemrograman: Java.
- Framework Backend: Spring Boot.
- Basis Data: PostgreSQL lokal atau Supabase.
- Perangkat Pendukung: IDE, Maven/Gradle, Git, dan REST API Client.

3.1.2.2. SPESIFIKASI PERANGKAT KERAS

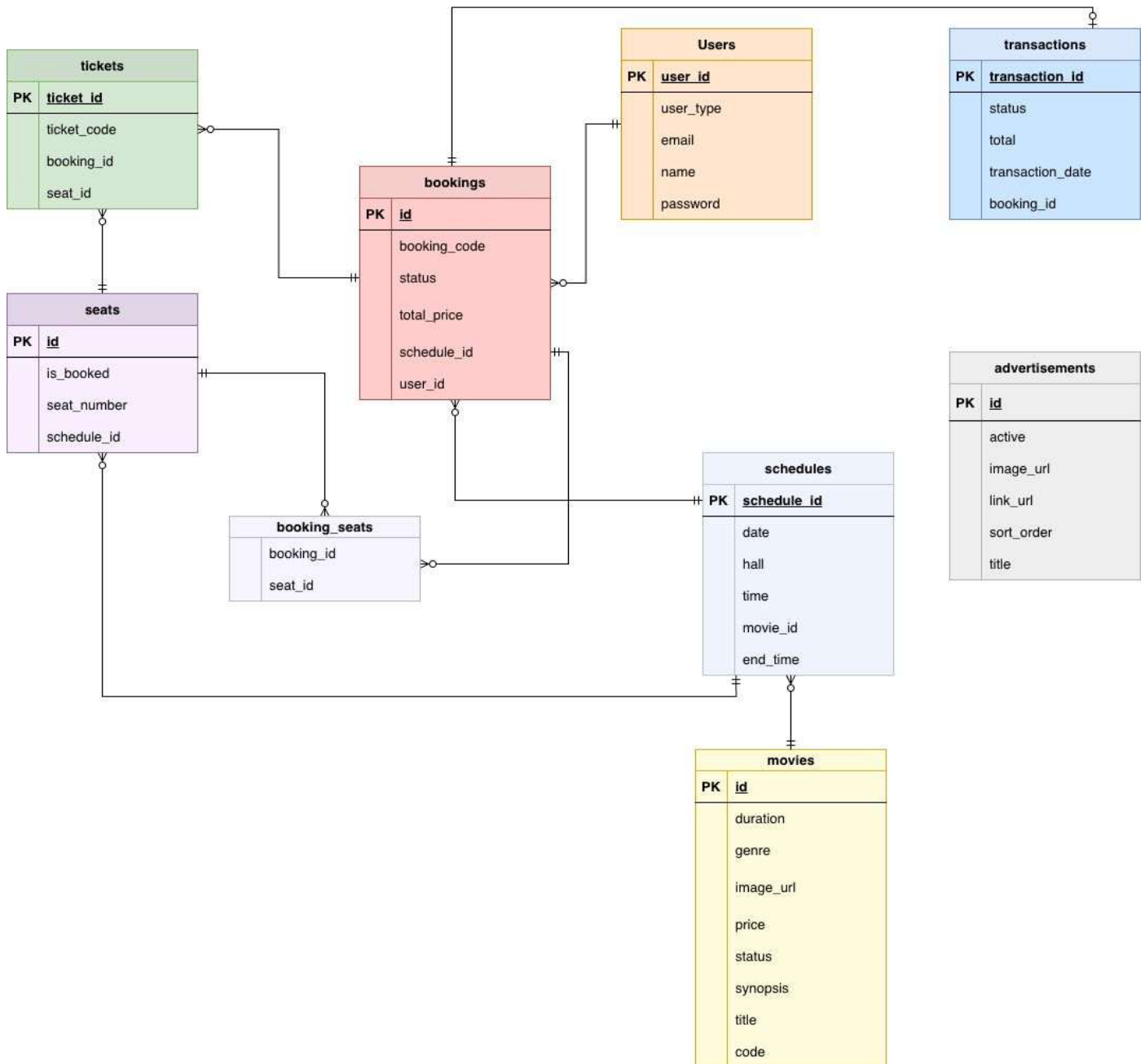
Spesifikasi perangkat keras yang disarankan adalah sebagai berikut:

- Prosesor minimal Intel Core i3 atau setara.
- Memori minimal 4 GB RAM.
- Penyimpanan minimal 10 GB ruang kosong.
- Perangkat uji berupa laptop atau komputer personal.
- Koneksi internet untuk kebutuhan integrasi dan pengujian API apabila diperlukan.

3.2 IMPLEMENTASI BASIS DATA

Implementasi basis data disusun berdasarkan ERD sistem *Cinema Ticket*. Setiap entitas dirancang sebagai tabel yang memiliki primary key. Relasi antartabel dihubungkan melalui foreign key sehingga alur data pemesanan tiket dapat tersimpan secara konsisten.

3.2.1 ENTITY RELATIONSHIP DIAGRAM (ERD)



Gambar 1: Entity Relationship Diagram terbaru sistem *Cinema Ticket*

ERD sistem terdiri atas entitas *users*, *movies*, *schedules*, *seats*, *bookings*, *booking seats*, *tickets*, *transactions*, dan *advertisements*. Entitas *users* menyimpan data pengguna dengan strategi

inheritance *SINGLE TABLE* melalui kolom diskriminator *user type*. Entitas *movies* menyimpan data film, entitas *schedules* menyimpan jadwal tayang, entitas *seats* menyimpan data kursi per jadwal, entitas *bookings* menyimpan data pemesanan, entitas *booking_seats* menyimpan relasi booking dengan kursi, entitas *tickets* menyimpan tiket yang diterbitkan, entitas *transactions* menyimpan data pembayaran, dan entitas *advertisements* menyimpan data iklan.

Alur penyimpanan data dimulai ketika pengguna pada tabel *users* memilih film dari tabel *movies*. Film memiliki jadwal pada tabel *schedules*. Setiap jadwal memiliki sejumlah kursi pada tabel *seats*. Ketika pengguna melakukan pemesanan, sistem membuat data pada tabel *bookings* yang mengacu pada *user_id* dan *schedule id*. Kursi yang dipilih dicatat pada tabel *booking_seats*. Setelah pemesanan berhasil, sistem dapat membuat tiket pada tabel *tickets* dan transaksi pembayaran pada tabel *transactions* yang mengacu pada *booking id*.

3.2.2 STRUKTUR BASIS DATA

Struktur basis data disusun berdasarkan atribut yang terlihat pada ERD. Berikut adalah penjelasan tabel utama pada sistem.

3.2.2.1. TABEL USER

Table 1: Tabel Entitas User

Atribut	Keterangan	Deskripsi
<i>user_id</i>	Primary Key	Identitas unik setiap pengguna.
<i>user_type</i>	Atribut	Jenis pengguna, misalnya customer atau admin.
<i>email</i>	Atribut	Email pengguna untuk proses autentikasi.
<i>name</i>	Atribut	Nama pengguna.
<i>password</i>	Atribut	Kata sandi pengguna.

3.2.2.2. TABEL MOVIES

Tabel *movies* memiliki primary key *id*. Atribut yang disimpan meliputi *duration*, *genre*, *image_url*, *price*, *status*, *synopsis*, *title*, dan *code*. Tabel ini menjadi sumber data utama untuk informasi film yang ditampilkan kepada customer dan dikelola oleh admin.

3.2.2.3. TABEL SCHEDULES

Tabel *schedules* memiliki primary key *schedule id*. Atributnya terdiri atas *date*, *hall*, *time*, *movie id*, dan *end.time*. Atribut *movie id* merupakan foreign key yang mengacu pada tabel *movies*. Relasi ini menunjukkan bahwa satu film dapat memiliki banyak jadwal tayang.

3.2.2.4. TABEL SEATS

Tabel *seats* memiliki primary key *id*. Atribut yang digunakan adalah *is booked*, *seat number*, dan *schedule id*. Atribut *schedule id* merupakan foreign key yang mengacu pada tabel *schedules*. Relasi ini menunjukkan bahwa setiap kursi terikat pada jadwal tayang tertentu.

3.2.2.5. TABEL BOOKINGS

Tabel *bookings* memiliki primary key *id*. Atribut yang disimpan adalah *booking code*, *status*, *total price*, *schedule id*, dan *user id*. Atribut *schedule id* menjadi foreign key ke tabel *schedules*, sedangkan *user id* menjadi foreign key ke tabel *users*. Relasi ini menunjukkan bahwa satu pengguna dapat membuat banyak booking dan satu jadwal dapat memiliki banyak booking.

3.2.2.6. TABEL BOOKING SEATS

Tabel *booking seats* berisi atribut *booking id* dan *seat id*. Keduanya berperan sebagai foreign key yang menghubungkan tabel *bookings* dan tabel *seats*. Tabel ini digunakan untuk menangani relasi banyak-ke-banyak antara booking dan kursi, karena satu booking dapat berisi lebih dari satu kursi dan satu kursi dicatat sebagai bagian dari booking tertentu.

3.2.2.7. TABEL TICKETS

Tabel *tickets* memiliki primary key *ticket id*. Atribut yang disimpan adalah *ticket code*, *booking id*, dan *seat id*. Atribut *booking id* mengacu pada tabel *bookings*, sedangkan *seat id* mengacu pada tabel *seats*. Tabel ini digunakan untuk menyimpan tiket yang diterbitkan setelah proses booking berhasil.

3.2.2.8. TABEL TRANSACTIONS

Tabel *transactions* memiliki primary key *transaction id*. Atribut yang disimpan adalah *status*, *total*, *transaction date*, dan *booking id*. Atribut *booking id* merupakan foreign key ke tabel *bookings*. Tabel ini mencatat status pembayaran dan total transaksi dari suatu booking.

3.2.2.9. TABEL ADVERTISEMENTS

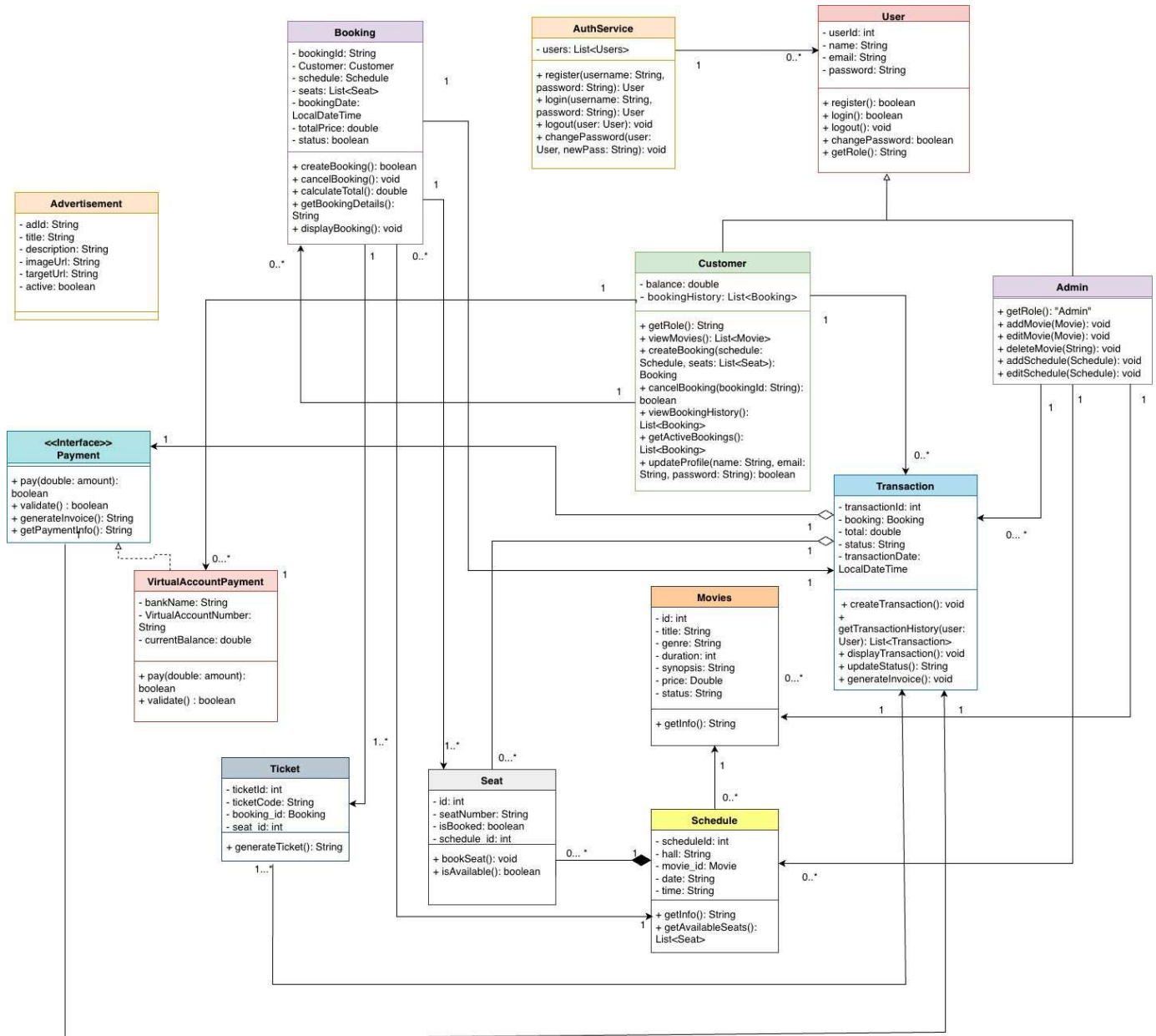
Tabel *advertisements* memiliki primary key *id*. Atributnya terdiri atas *active*, *image url*, *link url*, *sort order*, dan *title*. Tabel ini digunakan untuk menyimpan informasi iklan atau promosi yang dapat ditampilkan dalam aplikasi.

3.3 IMPLEMENTASI UML

Implementasi UML menjelaskan struktur kelas dan relasi antarkelas pada sistem *Cinema Ticket*. Diagram menunjukkan bahwa sistem menggunakan inheritance, interface, asosiasi, agregasi,

komposisi, serta encapsulation pada atribut kelas.

3.3.1 CLASS DIAGRAM



Gambar 2: Class Diagram terbaru sistem *Cinema Ticket*

Class Diagram menunjukkan bahwa *User* menjadi kelas induk bagi *Customer* dan *Admin*. Kelas *AuthService* berasosiasi dengan *User* karena menyimpan daftar pengguna dan menyediakan layanan autentikasi. Kelas *Customer* berasosiasi dengan *Movie*, *Schedule*, *Seat*, *Booking*, dan *Transaction* karena customer melakukan proses pemesanan. Kelas *Admin* berasosiasi dengan *Movie*, *Schedule*, *Seat*, dan *Advertisement* karena admin mengelola data utama sistem.

Interface *Payment* diimplementasikan oleh *VirtualAccountPayment*. Dengan demikian, method

pay(), *validate()*, *generateInvoice()*, dan *getPaymentInfo()* didefinisikan secara abstrak pada interface,

kemudian dijalankan secara konkret oleh kelas pembayaran virtual account. Kelas *Transaction* berhubungan dengan *Booking* dan pembayaran karena transaksi dibuat berdasarkan booking dan memiliki status pembayaran.

3.3.2 DESKRIPSI KELAS

Berikut adalah deskripsi kelas-kelas utama yang terdapat pada Class Diagram.

3.3.2.1. KELAS USER

Kelas *User* memiliki atribut private *userId*, *name*, *email*, dan *password*. Method yang tersedia adalah *register()*, *login()*, *logout()*, *changePassword()*, dan *getRole()*. Kelas ini menjadi superclass bagi *Customer* dan *Admin* dengan strategi JPA *SINGLE TABLE* dan discriminator *user type*. Encapsulation terlihat dari penggunaan atribut private sehingga data pengguna tidak diakses langsung dari luar kelas.

3.3.2.2. KELAS CUSTOMER

Kelas *Customer* mewarisi *User* dan memiliki atribut private *balance* serta *bookingHistory*. Method yang tersedia meliputi *getRole()*, *viewMovies()*, *createBooking()*, *cancelBooking()*, *viewBookingHistory()*, *getActiveBookings()*, dan *updateProfile()*. Kelas ini menunjukkan polymorphism melalui implementasi *getRole()* yang menghasilkan peran customer.

3.3.2.3. KELAS ADMIN

Kelas *Admin* juga mewarisi *User*. Method yang tersedia adalah *getRole()*, *addMovie()*, *updateMovie()*, *deleteMovie()*, *addSchedule()*, *updateSchedule()*, dan *viewAllTransactions()*. Kelas ini menunjukkan polymorphism karena memiliki method *getRole()* dengan hasil peran admin. Admin bertanggung jawab terhadap pengelolaan data film dan jadwal.

3.3.2.4. KELAS AUTHSERVICE

Kelas *AuthService* memiliki atribut *userRepository* dan *users: List<User>* dan method *register()*, *login()*, *logout()*, serta *changePassword()*. Kelas ini berfungsi sebagai layanan autentikasi dan manajemen akun. Relasinya dengan *User* menunjukkan bahwa satu layanan autentikasi dapat menangani banyak pengguna.

3.3.2.5. KELAS MOVIE

Kelas *Movie* memiliki atribut *id*, *title*, *code*, *genre*, *duration*, *synopsis*, *price*, *imageUrl*, dan *status*. Method *getInfo()* digunakan untuk menampilkan informasi film. Kelas ini berhubungan dengan *Schedule* karena satu film dapat memiliki beberapa jadwal tayang.

3.3.2.6. KELAS SCHEDULE

Kelas *Schedule* memiliki atribut *scheduleId*, *hall*, *movie*, *date*, *time*, dan *endTime*. Method *getInfo()* menampilkan detail jadwal, sedangkan *getAvailableSeats()* mengembalikan daftar kursi yang masih tersedia. Relasi dengan *Movie* menunjukkan bahwa jadwal tidak dapat berdiri sendiri tanpa film.

3.3.2.7. KELAS SEAT

Kelas *Seat* memiliki atribut *id*, *seatNumber*, *isBooked*, dan relasi *schedule* yang tersimpan sebagai foreign key *schedule id*. Method *bookSeat()* digunakan untuk mengubah status kursi, sedangkan *isAvailable()* memeriksa apakah kursi masih dapat dipesan. Kelas ini berhubungan dengan *Schedule* dan *Ticket*.

3.3.2.8. KELAS TRANSACTION

Kelas *Transaction* memiliki atribut *transactionId*, *booking*, *total*, *status*, dan *transactionDate*. Method yang tersedia meliputi *createTransaction()*, *getTransactionHistory()*, *displayTransaction()*, *updateStatus()*, dan *generateInvoice()*. Kelas ini menangani pencatatan transaksi yang berasal dari booking.

3.3.2.9. KELAS TICKET

Kelas *Ticket* memiliki atribut *ticketId*, *ticketCode*, serta relasi *booking* dan *seat* yang tersimpan sebagai foreign key *booking id* dan *seat id*. Method *generateTicket()* digunakan untuk membuat kode tiket. Relasi tiket dengan booking dan seat menunjukkan bahwa tiket merupakan hasil dari pemesanan kursi tertentu pada jadwal tertentu.

3.3.2.10. INTERFACE PAYMENT DAN KELAS VIRTUALACCOUNTPAYMENT

Interface *Payment* mendefinisikan method *pay()*, *validate()*, *generateInvoice()*, dan *getPaymentInfo()*. Kelas *VirtualAccountPayment* mengimplementasikan interface tersebut dan memiliki atribut *walletType*, *phoneNumber*, dan *walletBalance*. Implementasi interface ini merupakan bentuk abstraction dan polymorphism karena sistem dapat memperlakukan objek pembayaran melalui tipe *Payment* tanpa bergantung pada detail kelas pembayaran tertentu.

3.3.2.11. KELAS ADVERTISEMENT

Kelas *Advertisement* memiliki atribut *id*, *title*, *imageUrl*, *linkUrl*, *active*, dan *sortOrder*. Kelas ini digunakan untuk mengelola informasi promosi yang dapat ditampilkan pada aplikasi.

3.4 IMPLEMENTASI ANTARMUKA PENGGUNA (USER INTERFACE)

Implementasi antarmuka pengguna disusun berdasarkan fitur utama yang tersedia pada sistem. Antarmuka harus memudahkan customer dalam melakukan pemesanan tiket dan memudahkan admin dalam mengelola data sistem.

3.4.1 HALAMAN LOGIN

Halaman login digunakan oleh user untuk masuk ke sistem menggunakan email dan password. Proses login terhubung dengan kelas *AuthService* dan tabel *users*. Jika data valid, sistem mengarahkan pengguna sesuai perannya, yaitu customer atau admin.

3.4.2 HALAMAN DAFTAR FILM

Halaman daftar film menampilkan data dari kelas *Movie* dan tabel *movies*. Informasi yang ditampilkan meliputi judul, genre, durasi, sinopsis, harga, status, dan gambar film. Customer dapat memilih film untuk melihat jadwal tayang yang tersedia.

3.4.3 HALAMAN BOOKING DAN PEMILIHAN KURSI

Halaman booking digunakan customer untuk memilih jadwal dan kursi. Data jadwal berasal dari tabel *schedules*, sedangkan data kursi berasal dari tabel *seats*. Ketika kursi dipilih, sistem membuat data pada tabel *bookings* dan *booking seats*.

3.4.4 HALAMAN PEMBAYARAN DAN TIKET

Halaman pembayaran menampilkan informasi transaksi dan metode pembayaran virtual account. Proses pembayaran menggunakan interface *Payment* dan implementasi *VirtualAccountPayment*. Setelah pembayaran berhasil, sistem menghasilkan tiket melalui kelas *Ticket* dan menyimpan data pada tabel *tickets*.

4 PENGUJIAN SISTEM

4.1 Tujuan Pengujian

Pengujian sistem bertujuan untuk memastikan bahwa seluruh fitur utama aplikasi *Cinema Ticket* berjalan sesuai kebutuhan. Pengujian juga dilakukan untuk memastikan bahwa relasi antara kelas, proses bisnis, dan penyimpanan data telah berjalan konsisten.

4.2 Lingkungan Pengujian

Lingkungan pengujian mencakup perangkat lunak dan perangkat keras yang digunakan untuk menjalankan sistem serta memvalidasi fungsi utama.

4.2.1 Perangkat Lunak Pengujian

Perangkat lunak pengujian meliputi sistem operasi Windows/Linux/macOS, Java JDK, Spring Boot, Maven, Flutter SDK, Dart, PostgreSQL atau Supabase, Git, IDE, dan REST API Client.

4.2.2 Perangkat Keras Pengujian

Perangkat keras pengujian meliputi komputer atau laptop dengan prosesor minimal Intel Core i3 atau setara, RAM minimal 4 GB, penyimpanan minimal 10 GB, serta koneksi internet apabila sistem diuji dengan layanan eksternal.

4.3 Metode Pengujian

Metode pengujian yang digunakan adalah kombinasi Black Box Testing dan White Box Testing. Black Box Testing digunakan untuk memvalidasi alur fitur dari sudut pandang pengguna, sedangkan White Box Testing digunakan untuk memeriksa perilaku internal class backend berdasarkan method, kondisi percabangan, validasi, perubahan state objek, dan exception yang mungkin terjadi.

4.3.1 Black Box dan White Box Testing

Black Box Testing diterapkan pada fitur aplikasi seperti autentikasi, pengelolaan film, pengelolaan jadwal, pemilihan kursi, booking, pembayaran, dan penerbitan tiket. White Box Testing diterapkan pada class backend seperti *AuthService*, *BookingService*, *Booking*, *Customer*, *Admin*, *Movie*, *Schedule*, *Seat*, *Ticket*, *Transaction*, dan *VirtualAccountPayment*. Data pengujian dan hasil uji pada Bab 5 mengikuti file *tabel.hasil testing baru.md*.

4.3.2 Skenario Pengujian

Table 2: Identifikasi Pengujian

Kelas/Fitur Uji	Butir Uji
<i>AuthService</i>	Pengujian login, register, logout, dan perubahan sandi berdasarkan email maupun userId.

<i>BookingService</i>	Pengujian proses booking, validasi user, validasi jadwal, validasi kursi, saldo virtual account, dan pembuatan tiket.
<i>Booking</i>	Pengujian pembuatan booking, pembatalan booking, perhitungan total harga, dan detail booking.
<i>Customer</i>	Pengujian pembuatan booking oleh customer, pembatalan booking, riwayat booking, active booking, update profil, dan role customer.
<i>Admin</i>	Pengujian pengelolaan movie, schedule, riwayat transaksi, dan role admin.
<i>Movie</i>	Pengujian inisialisasi data film, informasi film, dan status default film.
<i>Schedule</i>	Pengujian informasi jadwal dan daftar kursi yang tersedia.
<i>Seat</i>	Pengujian perubahan status kursi melalui <i>bookSeat()</i> , <i>cancelSeat()</i> , dan <i>isAvailable()</i> .
<i>Ticket</i>	Pengujian pembuatan informasi tiket, relasi booking, dan kode tiket.
<i>Transaction</i>	Pengujian pembuatan transaksi, update status, dan invoice transaksi.
<i>VirtualAccountPayment</i>	Pengujian validasi data pembayaran, kecukupan saldo, invoice, dan informasi virtual account.

5 HASIL UJI

Hasil uji pada bab ini disusun berdasarkan file *tabel hasil testing baru.md*. Pengujian menggunakan pendekatan White Box Testing pada class backend dengan JUnit 5 dan Mockito, sehingga skenario, data input, expected output, hasil aktual, dan status pengujian mengikuti implementasi proyek *Cinema Ticket*.

5.1 Pengujian Autentikasi dan Manajemen Akun

5.1.1 Gambar Kode Uji Kelas AuthService

```
package cinema.service;

import cinema.model.Admin;
import cinema.model.Customer;
import cinema.model.User;
import cinema.repository.UserRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@DisplayName("Pengujian Layanan Autentikasi (AuthService)")
public class AuthServiceTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private AuthService authService;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
    }
}
```

Gambar 3: Dokumentasi kode uji kelas AuthService bagian 1

```

@Test
@DisplayName("Uji Login - Berhasil")
void ujillogin_Berhasil() {
    User pengguna = new Customer("Budi Santoso", "budi@gmail.com", "sandi123");
    when(userRepository.findByEmailIgnoreCase("budi@gmail.com")).thenReturn(Optional.of(pengguna));

    User hasil = authService.login("budi@gmail.com", "sandi123");

    assertNotNull(hasil);
    assertEquals("budi@gmail.com", hasil.getEmail());
}

@Test
@DisplayName("Uji Login - Email Tidak Ditemukan")
void ujillogin_EmailTidakDitemukan() {
    when(userRepository.findByEmailIgnoreCase("tidakada@gmail.com")).thenReturn(Optional.empty());
    when(userRepository.findByNameIgnoreCase("tidakada@gmail.com")).thenReturn(Optional.empty());

    User hasil = authService.login("tidakada@gmail.com", "sandi123");

    assertNull(hasil);
}

@Test
@DisplayName("Uji Login - Password Salah")
void ujillogin_PasswordSalah() {
    User pengguna = new Customer("Budi Santoso", "budi@gmail.com", "sandi123");
    when(userRepository.findByEmailIgnoreCase("budi@gmail.com")).thenReturn(Optional.of(pengguna));

    User hasil = authService.login("budi@gmail.com", "sandsalah");

    assertNull(hasil);
}

```

Gambar 4: Dokumentasi kode uji kelas AuthService bagian 2

```

@Test
@DisplayName("Uji Login - Email Null")
void ujiLogin_EmailNull() {
    when(userRepository.findByEmailIgnoreCase("")).thenReturn(Optional.empty());
    when(userRepository.findByNameIgnoreCase("")).thenReturn(Optional.empty());

    User hasil = authService.login(null, "sandi123");

    assertNull(hasil);
}

@Test
@DisplayName("Uji Register - Berhasil")
void ujiRegister_Berhasil() {
    when(userRepository.findByEmail("baru@gmail.com")).thenReturn(Optional.empty());
    User penggunaBaru = new Customer("Pengguna Baru", "baru@gmail.com", "sandi123");
    when(userRepository.save(any(User.class))).thenReturn(penggunaBaru);

    User hasil = authService.register("Pengguna Baru", "baru@gmail.com", "sandi123", false);

    assertNotNull(hasil);
    assertEquals("Customer", hasil.getRole());
    assertEquals("baru@gmail.com", hasil.getEmail());
}

@Test
@DisplayName("Uji Register - Email Duplikat")
void ujiRegister_EmailDuplikat() {
    when(userRepository.findByEmail("ada@gmail.com")).thenReturn(Optional.of(new Customer()));

    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        authService.register("Pengguna Baru", "ada@gmail.com", "sandi123", false);
    });

    assertEquals("Email sudah terdaftar", exception.getMessage());
}

```

Gambar 5: Dokumentasi kode uji kelas AuthService bagian 3

```

@Test
@DisplayName("Uji Logout - Tidak Ada Exception")
void ujiLogout() {
    User pengguna = new Customer();
    assertDoesNotThrow(() -> authService.logout(pengguna));
}

@Test
@DisplayName("Uji Ubah Password (By User ID) - Berhasil")
void ujiUbahPasswordBerdasarkanUserId_Berhasil() {
    User pengguna = new Customer("Pengguna", "user@gmail.com", "sandilama");
    when(userRepository.findById(1)).thenReturn(Optional.of(pengguna));

    boolean hasil = authService.changePassword(1, "sandilama", "sandibaru");

    assertTrue(hasil);
    assertEquals("sandibaru", pengguna.getPassword());
    verify(userRepository, times(1)).save(pengguna);
}

@Test
@DisplayName("Uji Ubah Password (By User ID) - Tidak Ditemukan")
void ujiUbahPasswordBerdasarkanUserId_TidakDitemukan() {
    when(userRepository.findById(99)).thenReturn(Optional.empty());

    boolean hasil = authService.changePassword(99, "sandilama", "sandibaru");

    assertFalse(hasil);
}

```

Gambar 6: Dokumentasi kode uji kelas AuthService bagian 4

```

@Test
@DisplayName("Uji Ubah Password (By User ID) - Password Lama Salah")
void ujiUbahPasswordBerdasarkanUserId_PasswordLamaSalah() {
    User pengguna = new Customer("Pengguna", "user@gmail.com", "sandilama");
    when(userRepository.findById(1)).thenReturn(Optional.of(pengguna));

    boolean hasil = authService.changePassword(1, "sandiSalah", "sandiBaru");

    Ctrl+L for Agent
    assertFalse(hasil);
}

@Test
@DisplayName("Uji Ubah Password (By Email) - Berhasil")
void ujiUbahPasswordBerdasarkanEmail_Berhasil() {
    User pengguna = new Customer("Pengguna", "user@gmail.com", "sandilama");
    when(userRepository.findByEmail("user@gmail.com")).thenReturn(Optional.of(pengguna));

    authService.changePassword("user@gmail.com", "sandilama", "sandiBaru");

    assertEquals("sandiBaru", pengguna.getPassword());
    verify(userRepository, times(1)).save(pengguna);
}

@Test
@DisplayName("Uji Ubah Password (By Email) - Tidak Ditemukan")
void ujiUbahPasswordBerdasarkanEmail_TidakDitemukan() {
    when(userRepository.findByEmail("tidakada@gmail.com")).thenReturn(Optional.empty());

    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        authService.changePassword("tidakada@gmail.com", "sandilama", "sandiBaru");
    });

    assertEquals("User tidak ditemukan", exception.getMessage());
}

```

Gambar 7: Dokumentasi kode uji kelas AuthService bagian 5

```

@Test
@DisplayName("Uji Ubah Password (By Email) - Password Lama Salah")
void ujiUbahPasswordBerdasarkanEmail_PasswordLamaSalah() {
    User pengguna = new Customer("Pengguna", "user@gmail.com", "sandilama");
    when(userRepository.findByEmail("user@gmail.com")).thenReturn(Optional.of(pengguna));

    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        authService.changePassword("user@gmail.com", "sandiSalah", "sandiBaru");
    });

    assertEquals("Password lama salah", exception.getMessage());
}

```

Gambar 8: Dokumentasi kode uji kelas AuthService bagian 6

5.1.2 Kasus Uji

Table 3: Kasus Uji Pengujian Autentikasi dan Manajemen Akun

Identifikasi	Skenario	Input Data	Expected Output
TC-AUTH-01	Login berhasil	email="budi@gmail.com", password="sandi123"	Return instance User (Customer)
TC-AUTH-02	Login gagal (email tidak ditemukan)	email="tidakada@gmail.com", password="sandi123"	Return null
TC-AUTH-03	Login gagal (password salah)	email="budi@gmail.com", password="sandsalah"	Return null
TC-AUTH-04	Login gagal (email null)	email=null, password="sandi123"	Return null
TC-AUTH-05	Register berhasil	name="Pengguna Baru", email="baru@gmail.com", password="sandi123", isAdmin=false	Return instance User (Customer) baru
TC-AUTH-06	Register gagal (email duplikat)	name="Pengguna Baru", email="ada@gmail.com", password="sandi123", isAdmin=false	Throw RuntimeException("Email sudah terdaftar")
TC-AUTH-07	Logout	Objek User (pengguna)	Tidak throw exception
TC-AUTH-08	Ubah Sandi (userId) - Berhasil	userId=1, oldPassword="sandiLama", newPassword="sandiBaru"	Return true, sandi ter-update
TC-AUTH-09	Ubah Sandi (userId) - User Tidak Ada	userId=99, oldPassword="sandiLama", newPassword="sandiBaru"	Return false
TC-AUTH-10	Ubah Sandi (userId) - Sandi Lama Salah	userId=1, oldPassword="sandiSalah", newPassword="sandiBaru"	Return false
TC-AUTH-11	Ubah Sandi (email) - Berhasil	email="user@gmail.com", oldPassword="sandiLama", newPassword="sandiBaru"	Berhasil mengubah sandi di DB
TC-AUTH-12	Ubah Sandi (email) - Email Tidak Ada	email="tidakada@gmail.com", oldPassword="sandiLama", newPassword="sandiBaru"	Throw RuntimeException("User tidak ditemukan")
TC-AUTH-13	Ubah Sandi (email) - Sandi Lama Salah	email="user@gmail.com", oldPassword="sandiSalah", newPassword="sandiBaru"	Throw RuntimeException("Password lama salah")

5.1.3 Hasil Uji

Table 4: Hasil Uji Pengujian Autentikasi dan Manajemen Akun

Identifikasi	Hasil yang Diharapkan	Hasil Sebenarnya	Status
TC-AUTH-01	Return instance User (Customer)	Mengembalikan objek User dengan email yang cocok	Lulus
TC-AUTH-02	Return null	Mengembalikan null tanpa exception	Lulus
TC-AUTH-03	Return null	Mengembalikan null karena kata sandi mismatch	Lulus
TC-AUTH-04	Return null	Mengembalikan null	Lulus
TC-AUTH-05	Return instance User (Customer) baru	Mengembalikan User tersimpan dengan Role "Customer"	Lulus

Identifikasi	Hasil yang Diharapkan	Hasil Sebenarnya	Status
TC-AUTH-06	Throw RuntimeException("Email sudah terdaftar")	Exception "Email sudah terdaftar" di-throw	Lulus
TC-AUTH-07	Tidak throw exception	Eksekusi method lancar	Lulus
TC-AUTH-08	Return true, sandi ter-update	Return true dan sandi terverifikasi baru	Lulus
TC-AUTH-09	Return false	Return false	Lulus
TC-AUTH-10	Return false	Return false	Lulus
TC-AUTH-11	Berhasil mengubah sandi di DB	userRepository.save() dipanggil 1 kali	Lulus
TC-AUTH-12	Throw RuntimeException("User tidak ditemukan")	Exception "User tidak ditemukan" di-throw	Lulus
TC-AUTH-13	Throw RuntimeException("Password lama salah")	Exception "Password lama salah" di-throw	Lulus

5.2 Pengujian Layanan Proses Booking

5.2.1 Gambar Kode Uji Layanan BookingService

```
package cinema.service;

import cinema.dto.BookingRequest;
import cinema.model.*;
import cinema.repository.*;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

import java.util.Arrays;
import java.util.Collections;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@DisplayName("Pengujian Layanan Pemesanan (BookingService)")
public class BookingServiceTest {

    @Mock private UserRepository userRepository;
    @Mock private ScheduleRepository scheduleRepository;
    @Mock private SeatRepository seatRepository;
    @Mock private BookingRepository bookingRepository;
    @Mock private TicketRepository ticketRepository;
    @Mock private TransactionRepository transactionRepository;

    @InjectMocks
    private BookingService bookingService;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
    }
}
```

Gambar 9: Dokumentasi kode uji layanan BookingService bagian 1

```

@Test
@DisplayName("Proses Booking - User Tidak Ditemukan")
void ujiProsesBooking_UserTidakDitemukan() {
    BookingRequest permintaan = new BookingRequest();
    permintaan.setUserId(1);
    when(userRepository.findById(1)).thenReturn(Optional.empty());

    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        bookingService.processBooking(permintaan);
    });

    assertEquals("User tidak ditemukan", exception.getMessage());
}

@Test
@DisplayName("Proses Booking - Jadwal Tidak Ditemukan")
void ujiProsesBooking_JadwalTidakDitemukan() {
    BookingRequest permintaan = new BookingRequest();
    permintaan.setUserId(1);
    permintaan.setScheduleId(1);
    when(userRepository.findById(1)).thenReturn(Optional.of(new Customer()));
    when(scheduleRepository.findById(1)).thenReturn(Optional.empty());

    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        bookingService.processBooking(permintaan);
    });

    assertEquals("Jadwal tidak ditemukan", exception.getMessage());
}

@Test
@DisplayName("Proses Booking - Kursi ID Tidak Ditemukan")
void ujiProsesBooking_KursiIdTidakDitemukan() {
    BookingRequest permintaan = new BookingRequest();
    permintaan.setUserId(1);
    permintaan.setScheduleId(1);
    permintaan.setSeatIds(Arrays.asList(1));

    when(userRepository.findById(1)).thenReturn(Optional.of(new Customer()));
    when(scheduleRepository.findById(1)).thenReturn(Optional.of(new Schedule()));
    when(seatRepository.findByIdForUpdate(1)).thenReturn(Optional.empty());

    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        bookingService.processBooking(permintaan);
    });

    assertEquals("Kursi ID 1 tidak ditemukan", exception.getMessage());
}

```

Gambar 10: Dokumentasi kode uji layanan BookingService bagian 2

```

@Test
@DisplayName("Proses Booking - Kursi Bukan Bagian Dari Jadwal")
void ujiProsesBooking_KursiBukanBagianJadwal() {
    BookingRequest permintaan = new BookingRequest();
    permintaan.setUserId(1);
    permintaan.setScheduleId(1);
    permintaan.setSeatIds(Arrays.asList(1));

    Schedule jadwalValid = new Schedule();
    jadwalValid.setScheduleId(1);

    Schedule jadwallain = new Schedule();
    jadwallain.setScheduleId(2);

    Seat kursi = new Seat();
    kursi.setSeatNumber("A1");
    kursi.setSchedule(jadwallain);

    when(userRepository.findById(1)).thenReturn(Optional.of(new Customer()));
    when(scheduleRepository.findById(1)).thenReturn(Optional.of(jadwalValid));
    when(seatRepository.findByIdForUpdate(1)).thenReturn(Optional.of(kursi));

    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        bookingService.processBooking(permintaan);
    });

    assertEquals("Kursi A1 bukan bagian dari jadwal ini", exception.getMessage());
}

```

Gambar 11: Dokumentasi kode uji layanan BookingService bagian 3

```

@Test
@DisplayName("Proses Booking - Kursi Sudah Dipesan")
void ujiProsesBooking_KursiSudahDipesan() {
    BookingRequest permintaan = new BookingRequest();
    permintaan.setUserId(1);
    permintaan.setScheduleId(1);
    permintaan.setSeatIds(Arrays.asList(1));

    Schedule jadwal = new Schedule();
    jadwal.setScheduleId(1);

    Seat kursi = new Seat();
    kursi.setSeatNumber("A1");
    kursi.setSchedule(jadwal);
    kursi.bookSeat(); // Sudah dipesan

    when(userRepository.findById(1)).thenReturn(Optional.of(new Customer()));
    when(scheduleRepository.findById(1)).thenReturn(Optional.of(jadwal));
    when(seatRepository.findByIdForUpdate(1)).thenReturn(Optional.of(kursi));

    Ctrl+L for Agent
    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        bookingService.processBooking(permintaan);
    });

    assertEquals("Kursi A1 sudah dipesan!", exception.getMessage());
}

@Test
@DisplayName("Proses Booking - Tidak Ada Kursi Yang Dipilih")
void ujiProsesBooking_TidakAdaKursiDipilih() {
    BookingRequest permintaan = new BookingRequest();
    permintaan.setUserId(1);
    permintaan.setScheduleId(1);

    when(userRepository.findById(1)).thenReturn(Optional.of(new Customer()));
    when(scheduleRepository.findById(1)).thenReturn(Optional.of(new Schedule()));

    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        bookingService.processBooking(permintaan);
    });

    assertEquals("Pilih minimal satu kursi", exception.getMessage());
}

```

Gambar 12: Dokumentasi kode uji layanan BookingService bagian 4

```

@Test
@DisplayName("Proses Booking - Saldo VA Tidak Cukup")
void ujiProsesBooking_SaldoTidakCukup() {
    BookingRequest permintaan = new BookingRequest();
    permintaan.setUserId(1);
    permintaan.setScheduleId(1);
    permintaan.setSeatIds(Arrays.asList(1));
    permintaan.setWalletBalance(10000.0); // Kurang dari harga tiket

    Schedule jadwal = new Schedule();
    jadwal.setScheduleId(1);
    jadwal.setMovie(new Movie("Film Keren", "Aksi", 120, "Sinopsis", 50000.0));

    Seat kursi = new Seat();
    kursi.setSeatNumber("A1");
    kursi.setSchedule(jadwal);

    when(userRepository.findById(1)).thenReturn(Optional.of(new Customer("Tes", "tes@gmail.com", "sandi")));
    when(scheduleRepository.findById(1)).thenReturn(Optional.of(jadwal));
    when(seatRepository.findByIdForUpdate(1)).thenReturn(Optional.of(kursi));

    RuntimeException exception = assertThrows(RuntimeException.class, () -> {
        bookingService.processBooking(permintaan);
    });

    assertEquals("Saldo Virtual Account tidak cukup", exception.getMessage());
}

```

Gambar 13: Dokumentasi kode uji layanan BookingService bagian 5

```

@Test
@DisplayName("Proses Booking - Berhasil Menggunakan Seat IDs")
void ujiProsesBooking_BerhasilDenganSeatIds() {
    BookingRequest permintaan = new BookingRequest();
    permintaan.setUserId(1);
    permintaan.setScheduleId(1);
    permintaan.setSeatIds(Arrays.asList(1));
    permintaan.setWalletBalance(100000.0);

    Schedule jadwal = new Schedule();
    jadwal.setScheduleId(1);
    jadwal.setMovie(new Movie("Film Keren", "Aksi", 120, "Sinopsis", 50000.0));

    Seat kursi = new Seat();
    kursi.setSeatNumber("A1");
    kursi.setSchedule(jadwal);

    when(userRepository.findById(1)).thenReturn(Optional.of(new Customer("Tes", "tes@gmail.com", "sandi")));
    when(scheduleRepository.findById(1)).thenReturn(Optional.of(jadwal));
    when(seatRepository.findByIdForUpdate(1)).thenReturn(Optional.of(kursi));
    when(bookingRepository.save(any(Booking.class))).thenAnswer(i -> i.getArguments()[0]);

    Ticket tiketHarapan = new Ticket();
    tiketHarapan.setTicketCode("TIKET-123");
    when(ticketRepository.saveAll(anyList())).thenReturn(Collections.singletonList(tiketHarapan));

    Ticket hasil = bookingService.processBooking(permintaan);

    assertNotNull(hasil);
    assertTrue(kursi.isBooked());
    verify(seatRepository, times(1)).saveAll(anyList());
    verify(transactionRepository, times(1)).save(any(Transaction.class));
}

```

Gambar 14: Dokumentasi kode uji layanan BookingService bagian 6

```

@Test
@DisplayName("Proses Booking - Berhasil Menggunakan Seat Numbers")
void ujiProsesBooking_BerhasilDenganSeatNumbers() {
    BookingRequest permintaan = new BookingRequest();
    permintaan.setUserId(1);
    permintaan.setScheduleId(1);
    permintaan.setSeatNumbers(Arrays.asList("A1"));
    permintaan.setWalletBalance(100000.0);

    Schedule jadwal = new Schedule();
    jadwal.setScheduleId(1);
    jadwal.setMovie(new Movie("Film Keren", "Aksi", 120, "Sinopsis", 50000.0));

    Seat kursi = new Seat();
    kursi.setSeatNumber("A1");
    kursi.setSchedule(jadwal);

    when(userRepository.findById(1)).thenReturn(Optional.of(new Customer("Tes", "tes@gmail.com", "sandi")));
    when(scheduleRepository.findById(1)).thenReturn(Optional.of(jadwal));
    when(seatRepository.findByScheduleAndSeatNumberForUpdate(jadwal, "A1")).thenReturn(kursi);
    when(bookingRepository.save(any(Booking.class))).thenReturn(i -> i.getArguments()[0]);

    Ticket tiketHarapan = new Ticket();
    tiketHarapan.setTicketCode("TIKET-123");
    when(ticketRepository.saveAll(anyList())).thenReturn(Collections.singletonList(tiketHarapan));

    Ticket hasil = bookingService.processBooking(permintaan);

    assertNotNull(hasil);
    assertTrue(kursi.isBooked());
}

```

Gambar 15: Dokumentasi kode uji layanan BookingService bagian 7

5.2.2 Kasus Uji

Table 5: Kasus Uji Pengujian Layanan Proses Booking

Identifikasi	Skenario	Input Data	Expected Output
TC-BOOKING-SVC-01	Proses Booking - User tidak ditemukan	userId=1 (tidak ada di DB)	Throw RuntimeException("User tidak ditemukan")
TC-BOOKING-SVC-02	Proses Booking - Jadwal tidak ditemukan	userId=1, scheduleId=1 (tidak ada di DB)	Throw RuntimeException("Jadwal tidak ditemukan")
TC-BOOKING-SVC-03	Proses Booking - Kursi tidak ditemukan	userId=1, scheduleId=1, seatIds=[1] (id tidak ada)	Throw RuntimeException("Kursi ID 1 tidak ditemukan")
TC-BOOKING-SVC-04	Proses Booking - Kursi bukan dari jadwal	userId=1, scheduleId=1, seatIds=[1] (beda jadwal)	Throw RuntimeException("Kursi A1 bukan bagian dari jadwal ini")
TC-BOOKING-SVC-05	Proses Booking - Kursi sudah dipesan	userId=1, scheduleId=1, seatIds=[1] (kursi penuh)	Throw RuntimeException("Kursi A1 sudah dipesan!")
TC-BOOKING-SVC-06	Proses Booking - Kursi belum dipilih	userId=1, scheduleId=1, seatIds=[]	Throw RuntimeException("Pilih minimal satu kursi")

Identifikasi	Skenario	Input Data	Expected Output
TC-BOOKING-SVC-07	Proses Booking - Saldo VA kurang	walletBalance=10000, total=50000	Throw RuntimeException("Saldo Virtual Account tidak cukup")
TC-BOOKING-SVC-08	Proses Booking - Sukses (pakai seatIds)	userId=1, scheduleId=1, seatIds=[1] saldo cukup	Return instance Ticket dan kursi ter-booking
TC-BOOKING-SVC-09	Proses Booking - Sukses (pakai seat-Numbers)	userId=1, scheduleId=1, seatNumbers=["A1"] saldo cukup	Return instance Ticket dan kursi ter-booking

5.2.3 Hasil Uji

Table 6: Hasil Uji Pengujian Layanan Proses Booking

Identifikasi	Hasil yang Diharapkan	Hasil Sebenarnya	Status
TC-BOOKING-SVC-01	Throw RuntimeException("User tidak ditemukan")	Exception ter-throw sesuai pesan	Lulus
TC-BOOKING-SVC-02	Throw RuntimeException("Jadwal tidak ditemukan")	Exception ter-throw sesuai pesan	Lulus
TC-BOOKING-SVC-03	Throw RuntimeException("Kursi ID 1 tidak ditemukan")	Exception ter-throw sesuai pesan	Lulus
TC-BOOKING-SVC-04	Throw RuntimeException("Kursi A1 bukan bagian dari jadwal ini")	Exception ter-throw sesuai pesan	Lulus
TC-BOOKING-SVC-05	Throw RuntimeException("Kursi A1 sudah dipesan!")	Exception ter-throw sesuai pesan	Lulus
TC-BOOKING-SVC-06	Throw RuntimeException("Pilih minimal satu kursi")	Exception ter-throw sesuai pesan	Lulus
TC-BOOKING-SVC-07	Throw RuntimeException("Saldo Virtual Account tidak cukup")	Exception ter-throw sesuai pesan	Lulus
TC-BOOKING-SVC-08	Return instance Ticket dan kursi ter-booking	Objek Ticket berhasil dikembalikan & isBooked() == true	Lulus
TC-BOOKING-SVC-09	Return instance Ticket dan kursi ter-booking	Objek Ticket berhasil dikembalikan & isBooked() == true	Lulus

5.3 Pengujian Kelas Booking

5.3.1 Gambar Kode Uji Kelas Booking

```
package cinema.model;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import java.util.Arrays;

import static org.junit.jupiter.api.Assertions.*;

@DisplayName("Pengujian Model Booking")
public class BookingTest {

    @Test
    @DisplayName("Uji Buat Booking")
    void ujiBuatBooking() {
        Booking pemesanan = new Booking();
        boolean hasil = pemesanan.createBooking();

        assertTrue(hasil);
        assertEquals("SUCCESS", pemesanan.getStatus());
    }
}
```

Gambar 16: Dokumentasi kode uji kelas Booking bagian 1

```

@Test
@DisplayName("Uji Buat Booking")
void ujiBuatBooking() {
    Booking pemesanan = new Booking();
    boolean hasil = pemesanan.createBooking();

    assertTrue(hasil);
    assertEquals("SUCCESS", pemesanan.getStatus());
}

@Test
@DisplayName("Uji Batalkan Booking - Dengan Kursi")
void ujiBatalBooking_DenganKursi() {
    Booking pemesanan = new Booking();
    Seat kursi1 = new Seat();
    kursi1.bookSeat();
    Seat kursi2 = new Seat();
    kursi2.bookSeat();

    pemesanan.setSeats(Arrays.asList(kursi1, kursi2));
    pemesanan.cancelBooking();

    assertEquals("CANCELLED", pemesanan.getStatus());
    assertFalse(kursi1.isBooked());
    assertFalse(kursi2.isBooked());
}

@Test
@DisplayName("Uji Batalkan Booking - Kursi Kosong (Null)")
void ujiBatalBooking_KursiNull() {
    Booking pemesanan = new Booking();
    pemesanan.setSeats(null);

    assertDoesNotThrow(() -> pemesanan.cancelBooking());
    assertEquals("CANCELLED", pemesanan.getStatus());
}

@Test
@DisplayName("Uji Hitung Total - Jadwal Null")
void ujiHitungTotal_JadwalNull() {
    Booking pemesanan = new Booking();
    pemesanan.setSchedule(null);
    assertEquals(0, pemesanan.calculateTotal());
}

```

Gambar 17: Dokumentasi kode uji kelas Booking bagian 2

```

@Test
@DisplayName("Uji Hitung Total - Film Null")
void ujiHitungTotal_FilmNull() {
    Booking pemesanan = new Booking();
    Schedule jadwal = new Schedule();
    jadwal.setMovie(null);
    pemesanan.setSchedule(jadwal);
    assertEquals(0, pemesanan.calculateTotal());
}

@Test
@DisplayName("Uji Hitung Total - Kursi Null")
void ujiHitungTotal_KursiNull() {
    Booking pemesanan = new Booking();
    Schedule jadwal = new Schedule();
    jadwal.setMovie(new Movie("Film Keren", "Aksi", 120, "Sinopsis", 50000));
    pemesanan.setSchedule(jadwal);
    pemesanan.setSeats(null);
    assertEquals(0, pemesanan.calculateTotal());
}

@Test
@DisplayName("Uji Hitung Total - Normal")
void ujiHitungTotal_Normal() {
    Booking pemesanan = new Booking();
    Schedule jadwal = new Schedule();
    jadwal.setMovie(new Movie("Film Keren", "Aksi", 120, "Sinopsis", 50000));
    pemesanan.setSchedule(jadwal);
    pemesanan.setSeats(Arrays.asList(new Seat(), new Seat()));

    assertEquals(100000, pemesanan.calculateTotal());
}

@Test
@DisplayName("Uji Dapatkan Detail Booking")
void ujiDapatkanDetailBooking() {
    Booking pemesanan = new Booking();
    pemesanan.setBookingCode("TIKET-123");
    pemesanan.setStatus("SUCCESS");
    pemesanan.setTotalPrice(100000);

    assertEquals("Booking TIKET-123 - SUCCESS - Total 100000.0", pemesanan.getBookingDetails());
}
}

```

Gambar 18: Dokumentasi kode uji kelas Booking bagian 3

5.3.2 Kasus Uji

Table 7: Kasus Uji Pengujian Kelas Booking

Identifikasi	Skenario	Input Data	Expected Output
TC-BOOKING-01	createBooking()	Objek Booking baru	Status berubah menjadi "SUCCESS", return true
TC-BOOKING-02	cancelBooking() (dengan daftar kursi)	Booking berisi list Seat	Status "CANCELLED", isBooked semua seat menjadi false

Identifikasi	Skenario	Input Data	Expected Output
TC-BOOKING-03	cancelBooking() (tanpa kursi)	Booking list Seat = null	Status "CANCELLED", tidak ada exception
TC-BOOKING-04	calculateTotal() - Jadwal null	schedule=null	Return 0
TC-BOOKING-05	calculateTotal() - Film null	movie=null pada schedule	Return 0
TC-BOOKING-06	calculateTotal() - Kursi null	seats=null	Return 0
TC-BOOKING-07	calculateTotal() - Normal	harga_film=50000, 2 kursi	Return 100000
TC-BOOKING-08	getBookingDetails()	bookingCode="TIKET-123", status="SUCCESS", total=100000	Return "Booking TIKET-123 - SUCCESS - Total 100000.0"

5.3.3 Hasil Uji

Table 8: Hasil Uji Pengujian Kelas Booking

Identifikasi	Hasil yang Diharapkan	Hasil Sebenarnya	Status
TC-BOOKING-01	Status berubah menjadi "SUCCESS", return true	Return true dan getStatus()=="SUCCESS"	Lulus
TC-BOOKING-02	Status "CANCELLED", isBooked semua seat menjadi false	Status="CANCELLED", isBooked()=="false" pada iterasi Seat	Lulus
TC-BOOKING-03	Status "CANCELLED", tidak ada exception	Eksekusi lancar, getStatus()=="CANCELLED"	Lulus
TC-BOOKING-04	Return 0	Nilai Total Price 0.0	Lulus
TC-BOOKING-05	Return 0	Nilai Total Price 0.0	Lulus
TC-BOOKING-06	Return 0	Nilai Total Price 0.0	Lulus
TC-BOOKING-07	Return 100000	Mengembalikan hasil kali 100000.0	Lulus
TC-BOOKING-08	Return "Booking TIKET-123 - SUCCESS - Total 100000.0"	String yang dikembalikan sesuai ekspektasi	Lulus

5.4 Pengujian Kelas Customer

5.4.1 Gambar Kode Uji Kelas Customer

```
package cinema.model;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import java.util.Arrays;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

@DisplayName("Pengujian Model Customer")
public class CustomerTest {
```

Gambar 19: Dokumentasi kode uji kelas Customer bagian 1

```

@Test
@DisplayName("Uji Pembuatan Booking Oleh Customer")
void ujiBuatBooking() {
    Customer pelanggan = new Customer();
    Schedule jadwal = new Schedule();
    jadwal.setMovie(new Movie("Film Keren", "Laga", 120, "Sinopsis", 50000));
    List<Seat> kursi = Arrays.asList(new Seat(), new Seat());

    Booking pemesanan = pelanggan.createBooking(jadwal, kursi);

    assertEquals("SUCCESS", pemesanan.getStatus());
    assertTrue(pelanggan.getBookingHistory().contains(pemesanan));
    assertEquals(100000, pemesanan.getTotalPrice());
}

@Test
@DisplayName("Uji Batalkan Booking - Jika Ditemukan")
void ujiBatalBooking_Ditemukan() {
    Customer pelanggan = new Customer();
    Booking pemesanan = new Booking();
    pemesanan.setBookingCode("TIKET-1");
    pelanggan.getBookingHistory().add(pemesanan);

    boolean hasil = pelanggan.cancelBooking("TIKET-1");

    assertTrue(hasil);
    assertEquals("CANCELLED", pemesanan.getStatus());
}

@Test
@DisplayName("Uji Batalkan Booking - Jika Tidak Ditemukan")
void ujiBatalBooking_TidakDitemukan() {
    Customer pelanggan = new Customer();

    boolean hasil = pelanggan.cancelBooking("TIKET-2");

    assertFalse(hasil);
}

```

Gambar 20: Dokumentasi kode uji kelas Customer bagian 2

```

@Test
@DisplayName("Uji Mendapatkan Booking Aktif")
void ujiDapatkanBookingAktif() {
    Customer pelanggan = new Customer();
    Booking pemesanan1 = new Booking(); pemesanan1.setStatus("SUCCESS");
    Booking pemesanan2 = new Booking(); pemesanan2.setStatus("CANCELLED");
    pelanggan.setBookingHistory(Arrays.asList(pemesanan1, pemesanan2));

    List<Booking> aktif = pelanggan.getActiveBookings();

    assertEquals(1, aktif.size());
    assertEquals("SUCCESS", aktif.get(0).getStatus());
}

@Test
@DisplayName("Uji Memperbarui Profil Customer")
void ujiUpdateProfil() {
    Customer pelanggan = new Customer();
    boolean hasil = pelanggan.updateProfile("Nama Baru", "baru@gmail.com", "sandibaru");

    assertTrue(hasil);
    assertEquals("Nama Baru", pelanggan.getName());
    assertEquals("baru@gmail.com", pelanggan.getEmail());
    assertEquals("sandibaru", pelanggan.getPassword());
}

@Test
@DisplayName("Uji Melihat Riwayat Booking")
void ujilihatRiwayatBooking() {
    Customer pelanggan = new Customer();
    Booking pemesanan = new Booking();
    pelanggan.setBookingHistory(Arrays.asList(pemesanan));

    assertEquals(1, pelanggan.viewBookingHistory().size());
}

@Test
@DisplayName("Uji Mendapatkan Peran Sebagai Customer")
void ujiDapatkanPeran() {
    Customer pelanggan = new Customer();
    assertEquals("Customer", pelanggan.getRole());
}
}

```

Gambar 21: Dokumentasi kode uji kelas Customer bagian 3

5.4.2 Kasus Uji

Table 9: Kasus Uji Pengujian Kelas Customer

Identifikasi	Skenario	Input Data	Expected Output
TC-CUST-01	createBooking() oleh Customer	Schedule dan List Seat	Booking baru di-history, status "SUCCESS"

Identifikasi	Skenario	Input Data	Expected Output
TC-CUST-02	cancelBooking() - Booking ada	bookingId valid ("TIKET-1")	Return true, status menjadi "CANCELLED"
TC-CUST-03	cancelBooking() - Booking tidak ada	bookingId tidak valid ("TIKET-2")	Return false
TC-CUST-04	getActiveBookings()	History berisi SUCCESS dan CANCELLED	Return hanya booking berstatus "SUCCESS"
TC-CUST-05	updateProfile()	nama="Nama Baru", email="baru@gmail.com", sandi="sandibaru"	Return true, data instance terupdate
TC-CUST-06	viewBookingHistory()	Instance dengan history 1 item	Return List berisi 1 item tersebut
TC-CUST-07	getRole()	Instance Customer	Return "Customer"

5.4.3 Hasil Uji

Table 10: Hasil Uji Pengujian Kelas Customer

Identifikasi	Hasil yang Diharapkan	Hasil Sebenarnya	Status
TC-CUST-01	Booking baru di-history, status "SUCCESS"	Size history 1, isinya obj Booking dengan total valid	Lulus
TC-CUST-02	Return true, status menjadi "CANCELLED"	Return true, getStatus()=="CANCELLED"	Lulus
TC-CUST-03	Return false	Return false tanpa exception	Lulus
TC-CUST-04	Return hanya booking berstatus "SUCCESS"	Size list yg direturn = 1, status = "SUCCESS"	Lulus
TC-CUST-05	Return true, data instance terupdate	Atribut class sesuai input baru	Lulus
TC-CUST-06	Return List berisi 1 item tersebut	Mengembalikan bookingHistory	Lulus
TC-CUST-07	Return "Customer"	Mengembalikan String "Customer"	Lulus

5.5 Pengujian Kelas Admin

5.5.1 Gambar Kode Uji Kelas Admin

```
package cinema.model;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

@DisplayName("Pengujian Model Admin")
public class AdminTest {
```

Gambar 22: Dokumentasi kode uji kelas Admin bagian 1

```

@Test
@DisplayName("Uji Menambah Film")
void ujiTambahFilm() {
    Admin pengelola = new Admin();
    Movie film = new Movie();
    pengelola.addMovie(film);

    assertTrue(pengelola.getManagedMovies().contains(film));
}

@Test
@DisplayName("Uji Update Film - Jika ID Sudah Ada")
void ujiUpdateFilm_IdAda() {
    Admin pengelola = new Admin();
    Movie filmLama = new Movie(); filmLama.setId(1); filmLama.setTitle("Lama");
    pengelola.addMovie(filmLama);

    Movie filmBaru = new Movie(); filmBaru.setId(1); filmBaru.setTitle("Baru");
    pengelola.updateMovie(filmBaru);

    assertEquals(1, pengelola.getManagedMovies().size());
    assertEquals("Baru", pengelola.getManagedMovies().get(0).getTitle());
}

@Test
@DisplayName("Uji Update Film - Jika ID Belum Ada (Baru)")
void ujiUpdateFilm_IdBaru() {
    Admin pengelola = new Admin();
    Movie film = new Movie(); film.setId(1);
    pengelola.updateMovie(film);

    assertEquals(1, pengelola.getManagedMovies().size());
}

@Test
@DisplayName("Uji Menghapus Film")
void ujiHapusFilm() {
    Admin pengelola = new Admin();
    Movie film = new Movie(); film.setId(1);
    pengelola.addMovie(film);

    pengelola.deleteMovie("1");

    assertTrue(pengelola.getManagedMovies().isEmpty());
}

```

Gambar 23: Dokumentasi kode uji kelas Admin bagian 2

```

@Test
@DisplayName("Uji Menambah Jadwal")
void ujiTambahJadwal() {
    Admin pengelola = new Admin();
    Schedule jadwal = new Schedule();
    pengelola.addSchedule(jadwal);

    assertTrue(pengelola.getManagedSchedules().contains(jadwal));
}

@Test
@DisplayName("Uji Update Jadwal - Jika ID Sudah Ada")
void ujiUpdateJadwal_IdAda() {
    Admin pengelola = new Admin();
    Schedule jadwal1 = new Schedule(); jadwal1.setScheduleId(1); jadwal1.setTime("10:00");
    pengelola.addSchedule(jadwal1);

    Schedule jadwal2 = new Schedule(); jadwal2.setScheduleId(1); jadwal2.setTime("12:00");
    pengelola.updateSchedule(jadwal2);

    assertEquals(1, pengelola.getManagedSchedules().size());
    assertEquals("12:00", pengelola.getManagedSchedules().get(0).getTime());
}

@Test
@DisplayName("Uji Update Jadwal - Jika ID Belum Ada (Baru)")
void ujiUpdateJadwal_IdBaru() {
    Admin pengelola = new Admin();
    Schedule jadwal = new Schedule(); jadwal.setScheduleId(1);
    pengelola.updateSchedule(jadwal);

    assertEquals(1, pengelola.getManagedSchedules().size());
}

@Test
@DisplayName("Uji Melihat Semua Transaksi")
void ujilihatSemuaTransaksi() {
    Admin pengelola = new Admin();
    Transaction transaksi = new Transaction();
    pengelola.getTransactionHistory().add(transaksi);

    assertEquals(1, pengelola.viewAllTransactions().size());
}

```

Gambar 24: Dokumentasi kode uji kelas Admin bagian 3

```

@Test
@DisplayName("Uji Mendapatkan Peran Sebagai Admin")
void ujiDapatkanPeran() {
    Admin pengelola = new Admin();
    assertEquals("Admin", pengelola.getRole());
}
}

```

Gambar 25: Dokumentasi kode uji kelas Admin bagian 4

5.5.2 Kasus Uji

Table 11: Kasus Uji Pengujian Kelas Admin

Identifikasi	Skenario	Input Data	Expected Output
TC-ADMIN-01	addMovie()	Objek Movie baru	Film masuk ke managedMovies
TC-ADMIN-02	updateMovie() - ID ada	Objek Movie dengan id sama, data beda	Film pada managedMovies terupdate di tempat
TC-ADMIN-03	updateMovie() - ID tidak ada	Objek Movie dengan id baru	Film ditambah ke list managedMovies
TC-ADMIN-04	deleteMovie()	id=1	Film dengan ID 1 terhapus dari managedMovies
TC-ADMIN-05	addSchedule()	Objek Schedule baru	Jadwal masuk ke managedSchedules
TC-ADMIN-06	updateSchedule() - ID ada	Objek Schedule dengan id sama	Jadwal di managedSchedules terupdate di tempat
TC-ADMIN-07	updateSchedule() - ID tidak ada	Objek Schedule dengan id baru	Jadwal ditambah ke managedSchedules
TC-ADMIN-08	viewAllTransactions()	Instance dengan history transaksi	Return transactionHistory
TC-ADMIN-09	getRole()	Instance Admin	Return "Admin"

5.5.3 Hasil Uji

Table 12: Hasil Uji Pengujian Kelas Admin

Identifikasi	Hasil yang Diharapkan	Hasil Sebenarnya	Status
TC-ADMIN-01	Film masuk ke managedMovies	managedMovies.size() == 1	Lulus
TC-ADMIN-02	Film pada managedMovies terupdate di tempat	Tidak terjadi duplikasi id, size list tetap 1	Lulus
TC-ADMIN-03	Film ditambah ke managedMovies	Film di-add, size = 1	Lulus

Identifikasi	Hasil yang Diharapkan	Hasil Sebenarnya	Status
TC-ADMIN-04	Film dengan ID 1 terhapus dari managedMovies	List menjadi kosong setelah delete() dipanggil	Lulus
TC-ADMIN-05	Jadwal masuk ke managedSchedules	managedSchedules.size() == 1	Lulus
TC-ADMIN-06	Jadwal di managedSchedules terupdate di tempat	Tidak duplikat ID, list update in-place	Lulus
TC-ADMIN-07	Jadwal ditambah ke managedSchedules	Jadwal di-add, size = 1	Lulus
TC-ADMIN-08	Return transactionHistory	Size 1 sesuai penambahan transient sebelumnya	Lulus
TC-ADMIN-09	Return "Admin"	Mengembalikan String "Admin"	Lulus

5.6 Pengujian Kelas Movie

5.6.1 Gambar Kode Uji Kelas Movie

```
package cinema.model;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

@DisplayName("Pengujian Model Movie (Film)")
public class MovieTest {

    @Test
    @DisplayName("Uji Pembuatan Melalui Konstruktor 5 Parameter")
    void ujiKonstruktor5Parameter() {
        Movie film = new Movie("Judul Film", "Laga", 120, "Deskripsi", 50000);

        assertEquals("Judul Film", film.getTitle());
        assertEquals("Laga", film.getGenre());
        assertEquals(120, film.getDuration());
        assertEquals("Deskripsi", film.getSynopsis());
        assertEquals(50000, film.getPrice());
    }

    @Test
    @DisplayName("Uji Mendapatkan Info Lengkap Film")
    void ujiDapatkanInfo() {
        Movie film = new Movie("Judul Film", "Laga", 120, "Deskripsi", 50000);
        assertEquals("Judul Film (Laga) - 120 mins", film.getInfo());
    }

    @Test
    @DisplayName("Uji Cek Status Default Saat Instansiasi")
    void ujiStatusDefault() {
        Movie film = new Movie();
        assertEquals("Now Showing", film.getStatus());
    }
}
```

Gambar 26: Dokumentasi kode uji kelas Movie bagian 1

5.6.2 Kasus Uji

Table 13: Kasus Uji Pengujian Kelas Movie

Identifikasi	Skenario	Input Data	Expected Output
TC-MOVIE-01	Konstruktor 5 Parameter	("Judul Film", "Laga", 120, "Deskripsi", 50000)	Atribut instance terisi semua dengan benar
TC-MOVIE-02	getInfo()	Instance film di atas	Return "Judul Film (Laga) - 120 mins"
TC-MOVIE-03	Status default	Instance Movie baru	Return "Now Showing"

5.6.3 Hasil Uji

Table 14: Hasil Uji Pengujian Kelas Movie

Identifikasi	Hasil yang Diharapkan	Hasil Sebenarnya	Status
TC-MOVIE-01	Atribut instance terisi semua dengan benar	title, genre, dll terinisialisasi non-null	Lulus
TC-MOVIE-02	Return "Judul Film (Laga) - 120 mins"	Format string output sesuai	Lulus
TC-MOVIE-03	Return "Now Showing"	Output return default = "Now Showing"	Lulus

5.7 Pengujian Kelas Schedule

5.7.1 Gambar Kode Uji Kelas Schedule

```
package cinema.model;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import java.util.Arrays;

import static org.junit.jupiter.api.Assertions.*;

@DisplayName("Pengujian Model Schedule (Jadwal)")
public class ScheduleTest {

    @Test
    @DisplayName("Uji Mendapatkan Info Jadwal Lengkap")
    void ujiDapatkanInfo() {
        Schedule jadwal = new Schedule(new Movie("Film Keren", "Laga", 120, "Sinopsis", 50000), "10:00");
        jadwal.setScheduleId(1);
        assertEquals("Schedule ID: 1, Movie: Film Keren, Time: 10:00", jadwal.getInfo());
    }

    @Test
    @DisplayName("Uji Mendapatkan Info - Kasus Film Null")
    void ujiDapatkanInfo_FilmNull() {
        Schedule jadwal = new Schedule(null, "10:00");
        jadwal.setScheduleId(1);
        assertEquals("Schedule ID: 1, Movie: N/A, Time: 10:00", jadwal.getInfo());
    }

    @Test
    @DisplayName("Uji Mendapatkan Daftar Kursi Yang Tersedia Saja")
    void ujiDapatkanKursiTersedia() {
        Schedule jadwal = new Schedule();
        Seat kursi1 = new Seat(); kursi1.setBooked(false);
        Seat kursi2 = new Seat(); kursi2.setBooked(true);
        Seat kursi3 = new Seat(); kursi3.setBooked(false);
        jadwal.setSeats(Arrays.asList(kursi1, kursi2, kursi3));

        assertEquals(2, jadwal.getAvailableSeats().size());
    }
}
```

Gambar 27: Dokumentasi kode uji kelas Schedule bagian 1

5.7.2 Kasus Uji

Table 15: Kasus Uji Pengujian Kelas Schedule

Identifikasi	Skenario	Input Data	Expected Output
TC-SCHED-01	getInfo() - Film tersedia	Movie="Film Keren", Time="10:00", ID=1	Return "Schedule ID: 1, Movie: Film Keren, Time: 10:00"
TC-SCHED-02	getInfo() - Film null	Movie=null, Time="10:00", ID=1	Return "Schedule ID: 1, Movie: N/A, Time: 10:00"

Identifikasi	Skenario	Input Data	Expected Output
TC-SCHED-03	getAvailableSeats()	List kursi (1 dipesan, 2 tersedia)	Return list berisi hanya 2 kursi yang tersedia

5.7.3 Hasil Uji

Table 16: Hasil Uji Pengujian Kelas Schedule

Identifikasi	Hasil yang Diharapkan	Hasil Sebenarnya	Status
TC-SCHED-01	Return "Schedule ID: 1, Movie: Film Keren, Time: 10:00"	Format string sesuai, property movie terbaca	Lulus
TC-SCHED-02	Return "Schedule ID: 1, Movie: N/A, Time: 10:00"	Format string handling null ("N/A") berhasil	Lulus
TC-SCHED-03	Return list berisi hanya 2 kursi yang tersedia	Size list = 2 (mem-filter isBooked = true)	Lulus

5.8 Pengujian Kelas Seat

5.8.1 Gambar Kode Uji Kelas Seat

```
package cinema.model;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

@DisplayName("Pengujian Model Seat (Kursi)")
public class SeatTest {

    @Test
    @DisplayName("Uji Pemesanan Kursi (isBooked menjadi True)")
    void ujiPesanKursi() {
        Seat kursi = new Seat();
        kursi.bookSeat();
        assertTrue(kursi.isBooked());
    }

    @Test
    @DisplayName("Uji Pembatalan Kursi (isBooked menjadi False)")
    void ujiBatalKursi() {
        Seat kursi = new Seat();
        kursi.bookSeat(); // true
        kursi.cancelSeat(); // false
        assertFalse(kursi.isBooked());
    }

    @Test
    @DisplayName("Uji Mengecek Status Ketersediaan Kursi")
    void ujiApakahTersedia() {
        Seat kursi = new Seat();
        assertTrue(kursi.isAvailable()); // default false jadi tersedia
        kursi.bookSeat();
        assertFalse(kursi.isAvailable());
    }
}
```

Gambar 28: Dokumentasi kode uji kelas Seat bagian 1

5.8.2 Kasus Uji

Table 17: Kasus Uji Pengujian Kelas Seat

Identifikasi	Skenario	Input Data	Expected Output
TC-SEAT-01	bookSeat()	Instance Seat baru	isBooked menjadi true
TC-SEAT-02	cancelSeat()	Seat berstatus dipesan (booked)	isBooked menjadi false
TC-SEAT-03	isAvailable()	Seat berstatus kosong	Return true

5.8.3 Hasil Uji

Table 18: Hasil Uji Pengujian Kelas Seat

Identifikasi	Hasil yang Diharapkan	Hasil Sebenarnya	Status
TC-SEAT-01	isBooked menjadi true	isBooked() menghasilkan true	Lulus
TC-SEAT-02	isBooked menjadi false	isBooked() menghasilkan false	Lulus
TC-SEAT-03	Return true	Method isAvailable() menghasilkan boolean true	Lulus

5.9 Pengujian Kelas Ticket

5.9.1 Gambar Kode Uji Kelas Ticket

```
package cinema.model;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

@DisplayName("Pengujian Model Ticket (Tiket)")
public class TicketTest {

    @Test
    @DisplayName("Uji Mencetak Tiket (Booking Tidak Null)")
    void ujiBuatTiket_BookingAda() {
        Ticket tiket = new Ticket();
        tiket.setTicketId(1);
        Booking pemesanan = new Booking();
        pemesanan.setId(100);
        tiket.setBooking(pemesanan);

        assertEquals("Ticket ID: 1, Booking ID: 100", tiket.generateTicket());
    }

    @Test
    @DisplayName("Uji Mencetak Tiket (Booking Null)")
    void ujiBuatTiket_BookingNull() {
        Ticket tiket = new Ticket();
        tiket.setTicketId(1);
        tiket.setBooking(null);

        assertEquals("Ticket ID: 1, Booking ID: N/A", tiket.generateTicket());
    }

    @Test
    @DisplayName("Uji Menetapkan dan Mendapatkan Kode Tiket")
    void ujiSetGetKodeTiket() {
        Ticket tiket = new Ticket();
        tiket.setTicketCode("TIKET-123");
        assertEquals("TIKET-123", tiket.getTicketCode());
    }
}
```

Gambar 29: Dokumentasi kode uji kelas Ticket bagian 1

5.9.2 Kasus Uji

Table 19: Kasus Uji Pengujian Kelas Ticket

Identifikasi	Skenario	Input Data	Expected Output
TC-TICK-01	generateTicket() - Booking ada	ticketId=1, bookingId=100	Return "Ticket ID: 1, Booking ID: 100"
TC-TICK-02	generateTicket() - Booking null	ticketId=1, booking=null	Return "Ticket ID: 1, Booking ID: N/A"
TC-TICK-03	set/getTicketCode()	code="TIKET-123"	Return "TIKET-123"

5.9.3 Hasil Uji

Table 20: Hasil Uji Pengujian Kelas Ticket

Identifikasi	Hasil yang Diharapkan	Hasil Sebenarnya	Status
TC-TICK-01	Return "Ticket ID: 1, Booking ID: 100"	Format code matching ID booking	Lulus
TC-TICK-02	Return "Ticket ID: 1, Booking ID: N/A"	Handling null-safe obj booking	Lulus
TC-TICK-03	Return "TIKET-123"	Method get() mengembalikan value setter	Lulus

5.10 Pengujian Kelas Transaction

5.10.1 Gambar Kode Uji Kelas Transaction

```
package cinema.model;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

@DisplayName("Pengujian Model Transaction (Transaksi)")
public class TransactionTest {

    @Test
    @DisplayName("Uji Pembuatan Transaksi Baru (Status Default PENDING)")
    void ujiBuatTransaksi() {
        Transaction transaksi = new Transaction();
        transaksi.createTransaction();

        assertEquals("PENDING", transaksi.getStatus());
        assertNotNull(transaksi.getTransactionDate());
    }

    @Test
    @DisplayName("Uji Pembaruan Status Transaksi")
    void ujiUpdateStatus() {
        Transaction transaksi = new Transaction();
        transaksi.updateStatus("SUCCESS");
        assertEquals("SUCCESS", transaksi.getStatus());
    }

    @Test
    @DisplayName("Uji Pembuatan Faktur (Booking Tersedia)")
    void ujiBuatFaktur_BookingAda() {
        Transaction transaksi = new Transaction();
        transaksi.setTransactionId(1);
        transaksi.setTotal(100000);
        Booking pemesanan = new Booking();
        pemesanan.setBookingCode("TIKET-1");
        transaksi.setBooking(pemesanan);

        assertEquals("Invoice 1 - Booking TIKET-1 - Total 100000.0", transaksi.generateInvoice());
    }

    @Test
    @DisplayName("Uji Pembuatan Faktur (Booking Null)")
    void ujiBuatFaktur_BookingNull() {
        Transaction transaksi = new Transaction();
        transaksi.setTransactionId(1);
        transaksi.setTotal(100000);
        transaksi.setBooking(null);

        assertEquals("Invoice 1 - Booking N/A - Total 100000.0", transaksi.generateInvoice());
    }
}
```

Gambar 30: Dokumentasi kode uji kelas Transaction bagian 1

5.10.2 Kasus Uji

Table 21: Kasus Uji Pengujian Kelas Transaction

Identifikasi	Skenario	Input Data	Expected Output
TC-TRANS-01	createTransaction()	Instance baru	Status "PENDING", transactionDate tidak null
TC-TRANS-02	updateStatus()	status="SUCCESS"	Status berubah menjadi "SUCCESS"
TC-TRANS-03	generateInvoice() - Booking ada	id=1, total=100000, code="TIKET-1"	Return "Invoice 1 - Booking TIKET-1 - Total 100000.0"
TC-TRANS-04	generateInvoice() - Booking null	id=1, total=100000, booking=null	Return "Invoice 1 - Booking N/A - Total 100000.0"

5.10.3 Hasil Uji

Table 22: Hasil Uji Pengujian Kelas Transaction

Identifikasi	Hasil yang Diharapkan	Hasil Sebenarnya	Status
TC-TRANS-01	Status "PENDING", transactionDate tidak null	Inisialisasi DateTime dan string enum PENDING	Lulus
TC-TRANS-02	Status berubah menjadi "SUCCESS"	Status terupdate sempurna	Lulus
TC-TRANS-03	Return "Invoice 1 - Booking TIKET-1 - Total 100000.0"	Invoice string mapping property inner valid	Lulus
TC-TRANS-04	Return "Invoice 1 - Booking N/A - Total 100000.0"	Fallback N/A untuk entity booking null	Lulus

5.11 Pengujian Kelas VirtualAccountPayment

5.11.1 Gambar Kode Uji Kelas VirtualAccountPayment

```
package cinema.payment;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

@DisplayName("Pengujian Virtual Account Payment (Pembayaran VA)")
public class VirtualAccountPaymentTest {

    @Test
    @DisplayName("Uji Validasi Virtual Account")
    void ujiValidasi() {
        VirtualAccountPayment pembayaran1 = new VirtualAccountPayment("Bank BCA", "0812", 100000);
        assertTrue(pembayaran1.validate());

        VirtualAccountPayment pembayaran2 = new VirtualAccountPayment(null, "0812", 100000);
        assertFalse(pembayaran2.validate());

        VirtualAccountPayment pembayaran3 = new VirtualAccountPayment("", "0812", 100000);
        assertFalse(pembayaran3.validate());
    }

    @Test
    @DisplayName("Uji Proses Pembayaran (Cek Potongan Saldo)")
    void ujiBayar() {
        VirtualAccountPayment pembayaran = new VirtualAccountPayment("Bank BCA", "0812", 100000);

        assertTrue(pembayaran.pay(50000));
        assertFalse(pembayaran.pay(100000)); // Hanya tersisa 50000
    }

    @Test
    @DisplayName("Uji Cetak Faktur VA")
    void ujiBuatFaktur() {
        VirtualAccountPayment pembayaran = new VirtualAccountPayment("Bank BCA", "0812", 100000);
        assertEquals("Invoice for Virtual Account Bank BCA (0812)", pembayaran.generateInvoice());
    }

    @Test
    @DisplayName("Uji Dapatkan Info Pembayaran VA")
    void ujiDapatkanInfoPembayaran() {
        VirtualAccountPayment pembayaran = new VirtualAccountPayment("Bank BCA", "0812", 100000);
        assertEquals("Bank BCA - 0812", pembayaran.getPaymentInfo());
    }
}
```

Gambar 31: Dokumentasi kode uji kelas VirtualAccountPayment bagian 1

5.11.2 Kasus Uji

Table 23: Kasus Uji Pengujian Kelas VirtualAccountPayment

Identifikasi	Skenario	Input Data	Expected Output
TC-VA-01	validate() - Valid	wallet="Bank BCA", phone="0812"	Return true
TC-VA-02	validate() - Tidak valid	wallet=null atau kosong	Return false
TC-VA-03	pay() - Saldo Cukup	balance=100000, pay=50000	Return true, saldo berkurang
TC-VA-04	pay() - Saldo Kurang	balance=50000, pay=100000	Return false
TC-VA-05	generateInvoice()	wallet="Bank BCA", phone="0812"	Return "Invoice for Virtual Account Bank BCA (0812)"
TC-VA-06	getPaymentInfo()	wallet="Bank BCA", phone="0812"	Return "Bank BCA - 0812"

5.11.3 Hasil Uji

Table 24: Hasil Uji Pengujian Kelas VirtualAccountPayment

Identifikasi	Hasil yang Diharapkan	Hasil Sebenarnya	Status
TC-VA-01	Return true	Mengembalikan boolean true	Lulus
TC-VA-02	Return false	Mengembalikan boolean false karena null/kosong	Lulus
TC-VA-03	Return true, saldo berkurang	Mengurangi state objek secara virtual	Lulus
TC-VA-04	Return false	Transaksi ditolak saldo kurang, return false	Lulus
TC-VA-05	Return "Invoice for Virtual Account Bank BCA (0812)"	Mapping string getter	Lulus
TC-VA-06	Return "Bank BCA - 0812"	Output properti	Lulus

5.12 Bukti Running Testing

Subbab ini menampilkan bukti eksekusi pengujian backend yang menunjukkan bahwa rangkaian test berhasil dijalankan.

```

[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running cinema.model.AdminTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.098 s -- in cinema.model.AdminTest
[INFO] Running cinema.model.BookingTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.025 s -- in cinema.model.BookingTest
[INFO] Running cinema.model.CustomerTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.023 s -- in cinema.model.CustomerTest
[INFO] Running cinema.model.MovieTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.011 s -- in cinema.model.MovieTest
[INFO] Running cinema.model.ScheduleTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.011 s -- in cinema.model.ScheduleTest
[INFO] Running cinema.model.SeatTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.010 s -- in cinema.model.SeatTest
[INFO] Running cinema.model.TicketTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 s -- in cinema.model.TicketTest
[INFO] Running cinema.model.TransactionTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.033 s -- in cinema.model.TransactionTest
[INFO] Running cinema.payment.VirtualAccountPaymentTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.019 s -- in cinema.payment.VirtualAccountPaymentTest
[INFO] Running cinema.service.AuthServiceTest

```

Gambar 32: Bukti running testing bagian 1

```

[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.088 s -- in cinema.service.AuthServiceTest
[INFO] Running cinema.service.BookingServiceTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.308 s -- in cinema.service.BookingServiceTest
[INFO] Results:
[INFO] Tests run: 66, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 4.903 s
[INFO] Finished at: 2026-06-16T20:41:09+07:00
[INFO]

```

Gambar 33: Bukti running testing bagian 2

6 PENUTUP

Berdasarkan analisis UML Class Diagram dan ERD, sistem *Cinema Ticket* telah dirancang dengan struktur yang sesuai untuk aplikasi pemesanan tiket bioskop. Dari sisi OOP, sistem menerapkan inheritance melalui pewarisan *User* kepada *Customer* dan *Admin*, encapsulation melalui atribut *private* pada kelas, polymorphism melalui method *getRole()* dan implementasi interface pembayaran, serta abstraction melalui interface *Payment*. Dari sisi basis data, sistem memiliki entitas yang saling terhubung secara relasional sehingga alur data pengguna, film, jadwal, kursi, booking, tiket, dan transaksi dapat disimpan secara konsisten.

Saran pengembangan sistem ke depan adalah menambahkan metode pembayaran lain dengan tetap menggunakan interface *Payment*, meningkatkan keamanan penyimpanan password, menambahkan validasi transaksi secara real-time, serta mengembangkan antarmuka pengguna agar lebih interaktif dan mudah digunakan.

7 DAFTAR PUSTAKA

1. Booch, G., Rumbaugh, J., dan Jacobson, I. *The Unified Modeling Language User Guide*. Addison-Wesley.
2. Oracle. *The Java Tutorials: Object-Oriented Programming Concepts*.
3. Silberschatz, A., Korth, H. F., dan Sudarshan, S. *Database System Concepts*. McGraw-Hill.
4. Sommerville, I. *Software Engineering*. Pearson.

8 LAMPIRAN

8.1 Lampiran A Pembagian Tugas

Table 25: Pembagian Tugas Anggota Kelompok

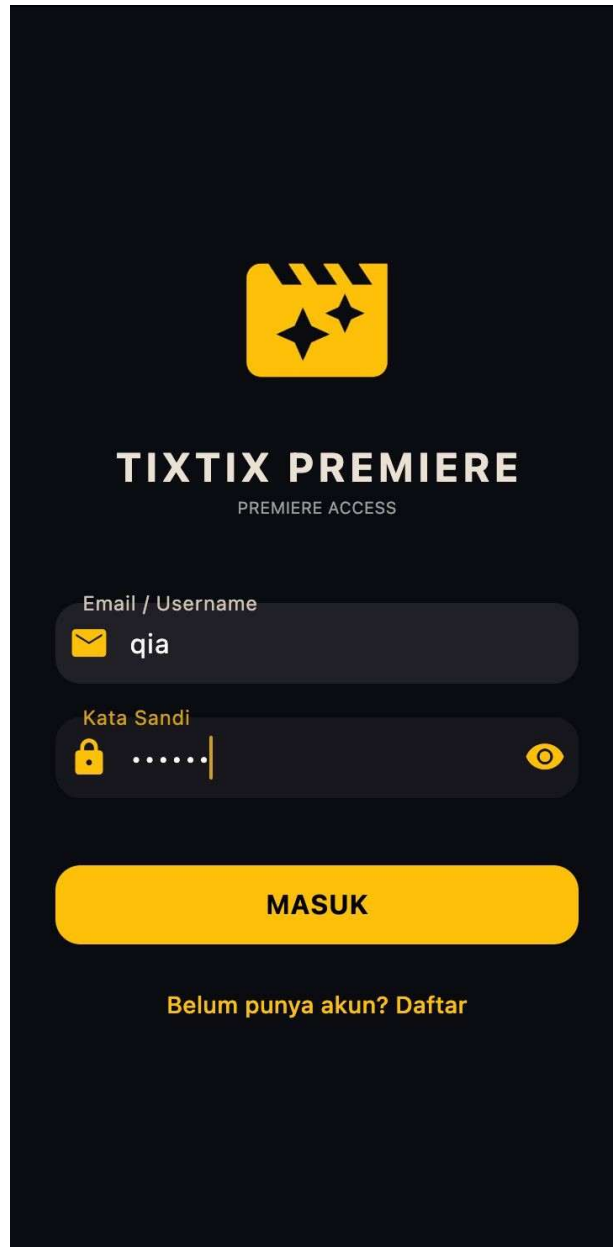
NIM	Nama	Tugas	Kontribusi (100%)
103012400028	Irzi Arinta Dwi Ponilan	Bab 1 laporan, fitur login/register, validasi user, dan koneksi autentikasi ke database	20%
103012400213	Tianisa Sanipar	Bab 2 laporan, fitur CRUD data film, pengelolaan judul, genre, durasi, rating, dan sinopsis film	20%
103012400266	Shakira Bilqis Sarwahita	Analisis kebutuhan sistem, ERD, struktur database, relasi tabel, dan koneksi MySQL	20%
103012300331	Siti Alqia Tonggiroh	UML, class diagram, fitur jadwal tayang, studio, pemilihan kursi, dan pengecekan ketersediaan kursi	20%
103012300133	Nadine Nafeesa Setyawan	Pengujian sistem, hasil uji, penutup, fitur pemesanan tiket, pembayaran, dan cetak/lihat tiket	20%
	Total		100%

8.2 Lampiran B Source Code Penting

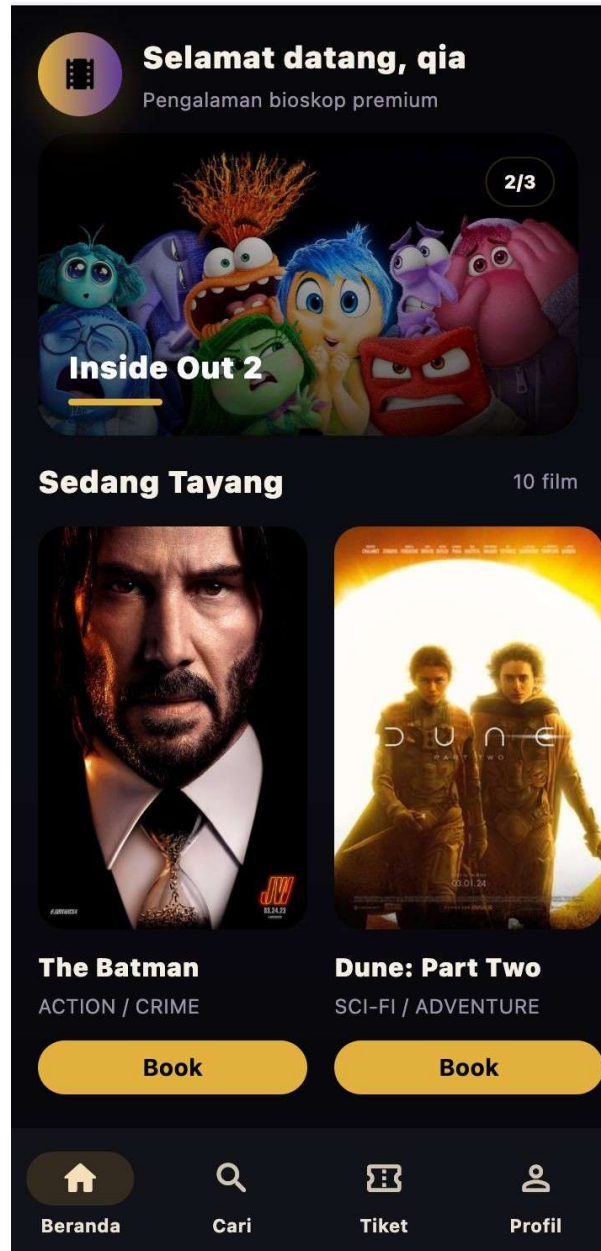
Source code: <https://github.com/arinponilan/cinema-ticket.git>

8.3 Lampiran C Screenshot Sistem

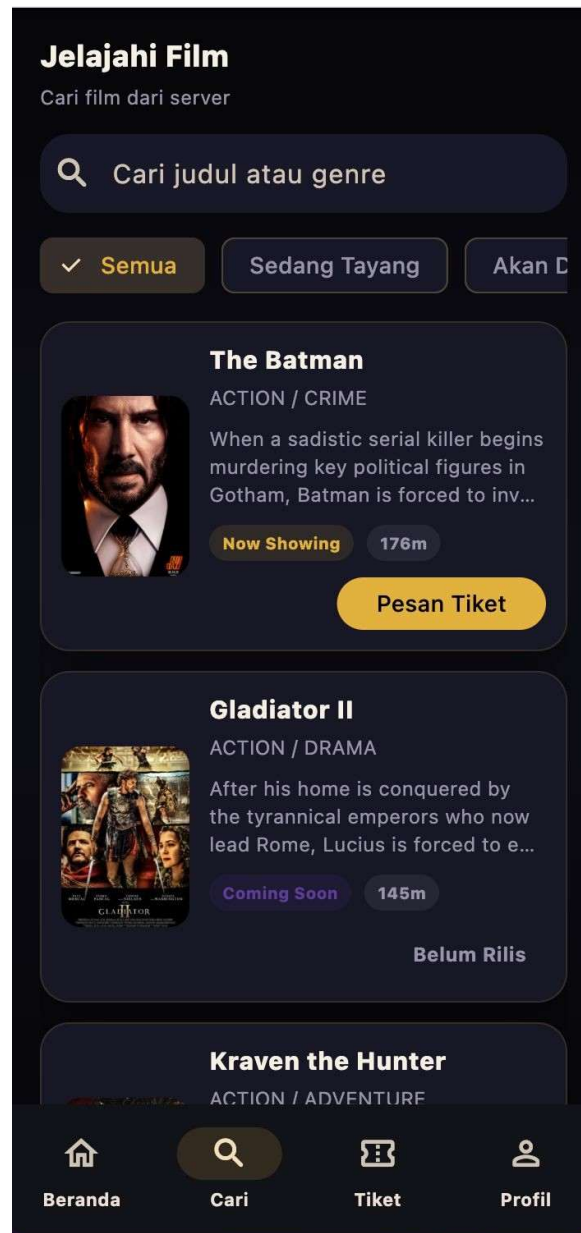
Bagian ini berisi screenshot sistem yang disusun sesuai alur penggunaan aplikasi, dimulai dari login, halaman customer, proses pemilihan kursi dan pembayaran, daftar tiket, profil, hingga halaman admin.



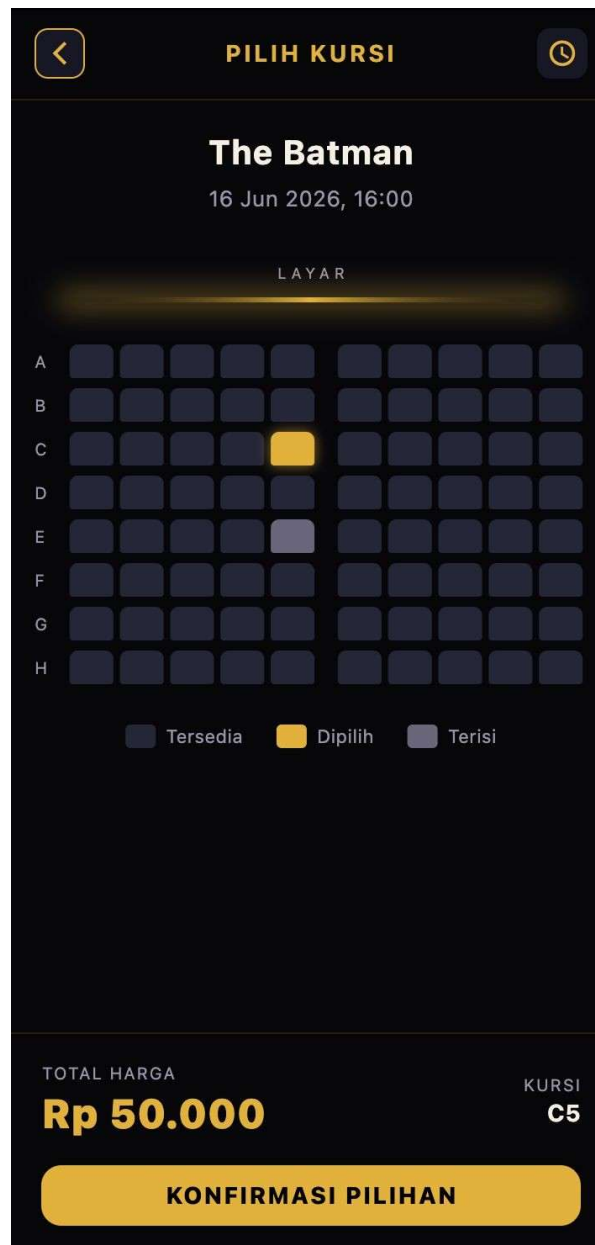
Gambar 34: Halaman login user



Gambar 35: Dashboard customer



Gambar 36: Halaman pencarian film



Gambar 37: Halaman pemilihan kursi customer

 **RINGKASAN PESANAN**



The Batman
ACTION / CRIME
🕒 16 Jun 2026, 16:00

METODE PEMBAYARAN



Virtual Account
Bayar via transfer bank

☐



QRIS
Bayar pakai aplikasi apa saja

☒

Kursi	C5
Tiket	1 x Rp 45.000
Subtotal	Rp 45.000
Biaya Layanan	Rp 5.000
Total Pembayaran	Rp 50.000

BAYAR

Gambar 38: Halaman pembayaran customer



Pembayaran

Total Pembayaran

Rp 50.000

Batas Pembayaran

00 : 14 : 42

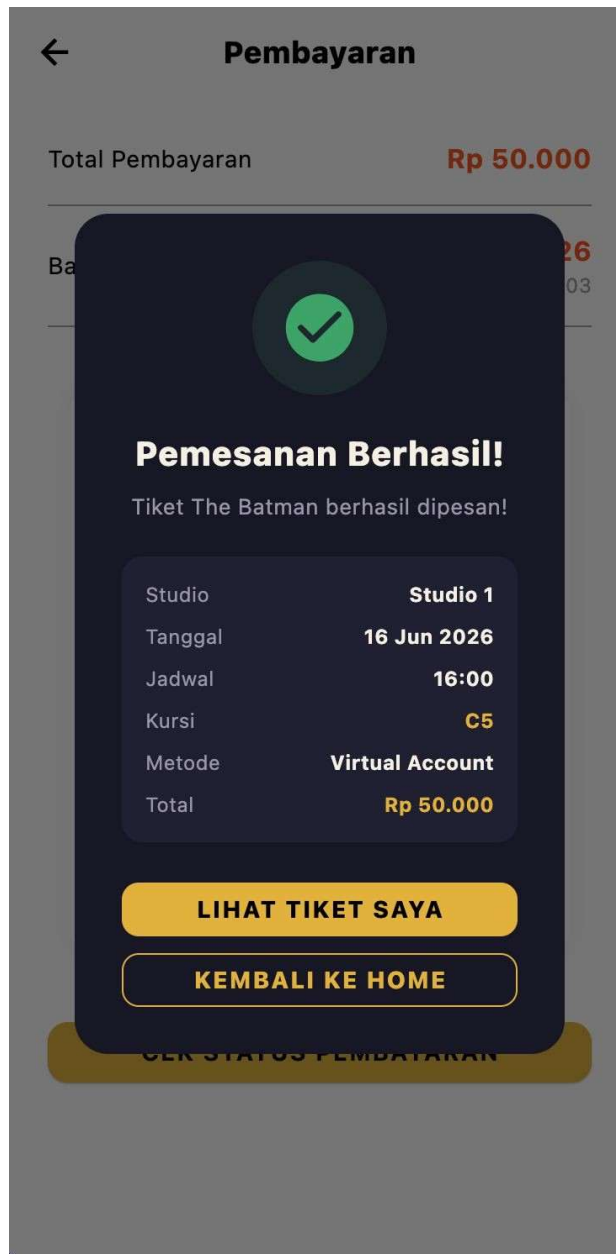
Jatuh tempo 16 Jun 2026, 22:03



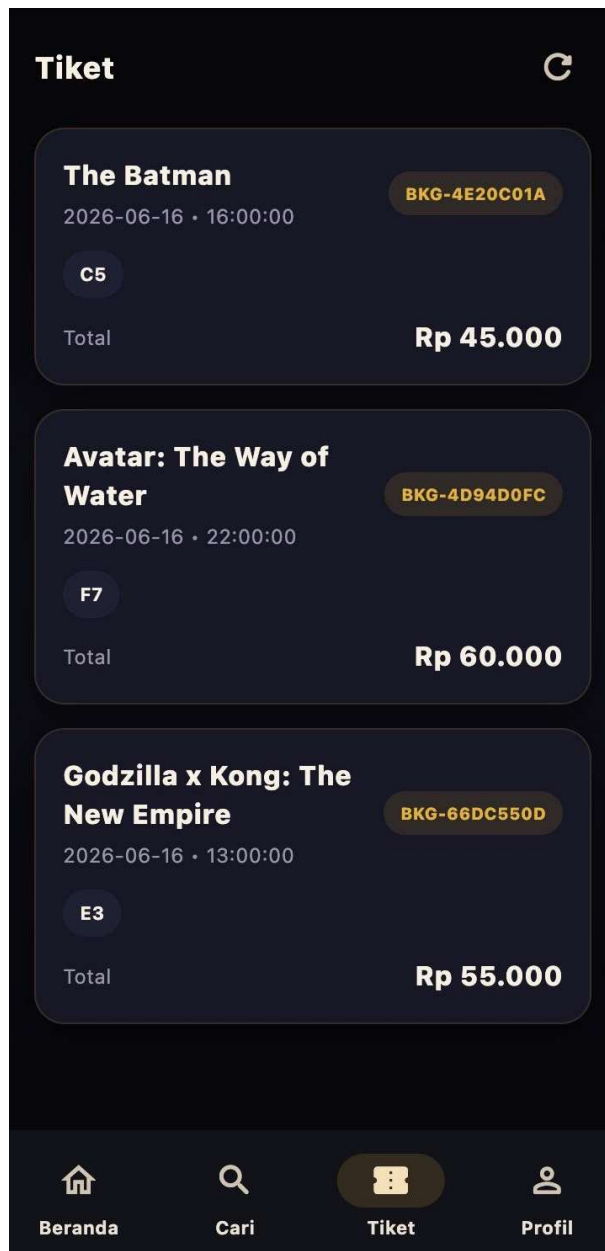
NMID:ID2025444802321

CEK STATUS PEMBAYARAN

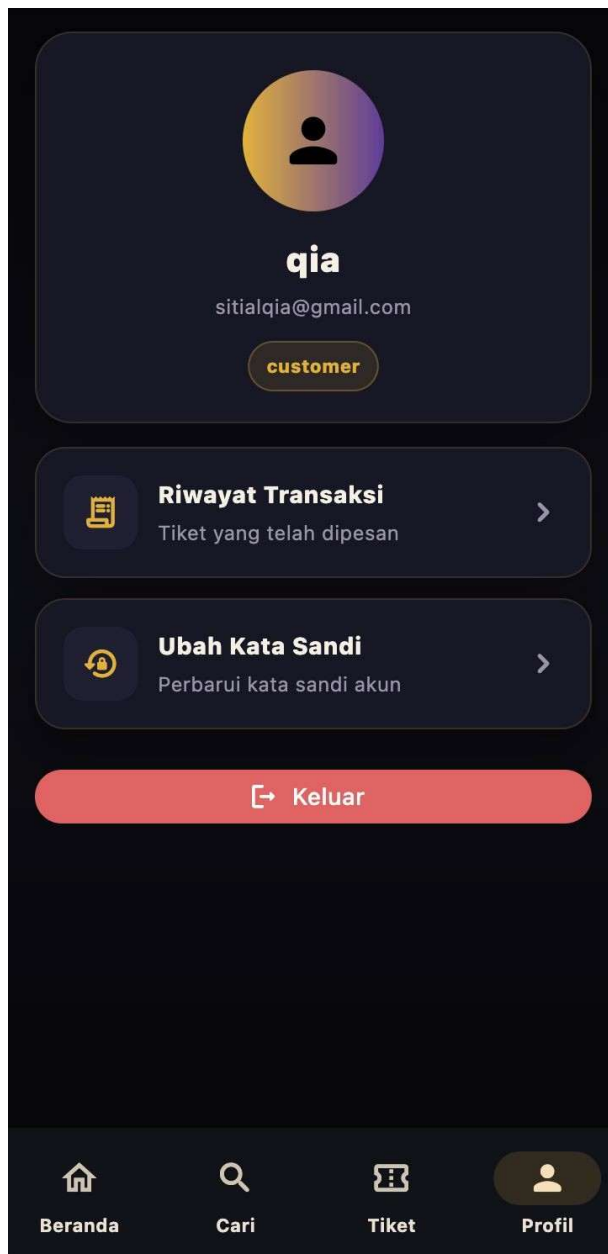
Gambar 39: Halaman pembayaran QRIS



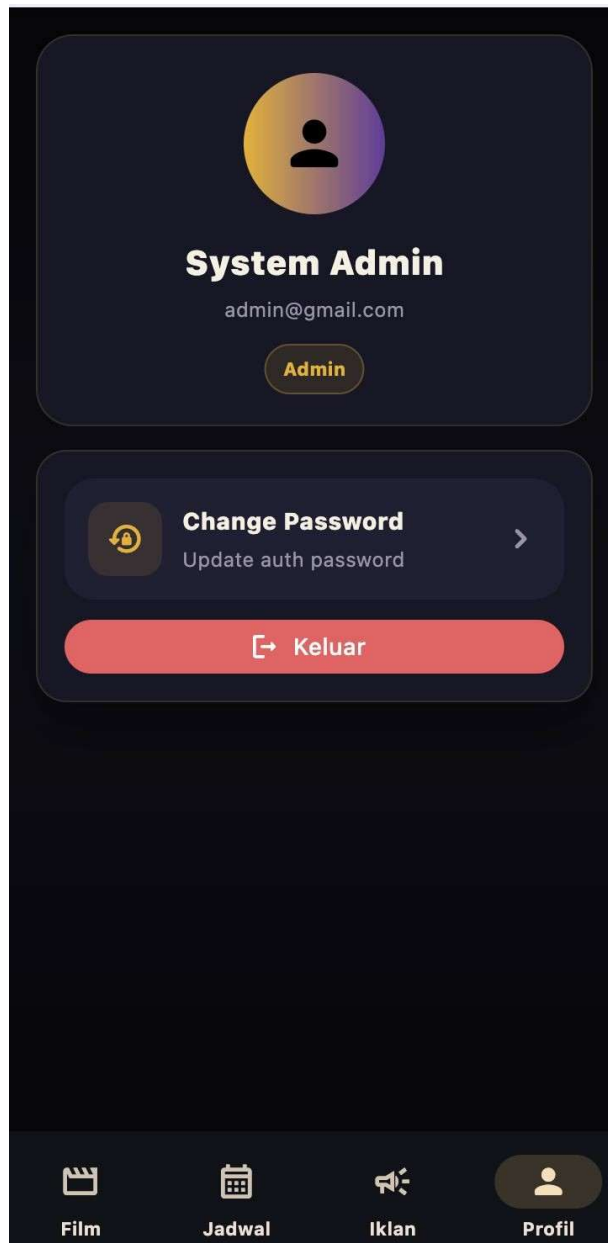
Gambar 40: Invoice pembayaran berhasil



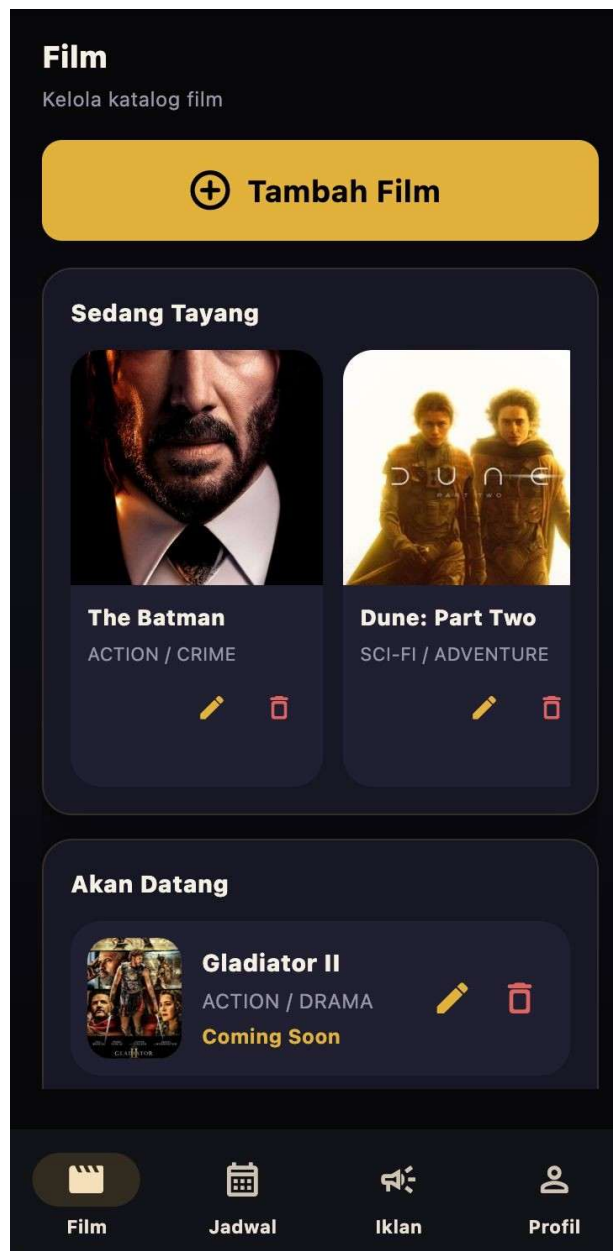
Gambar 41: Daftar tiket customer



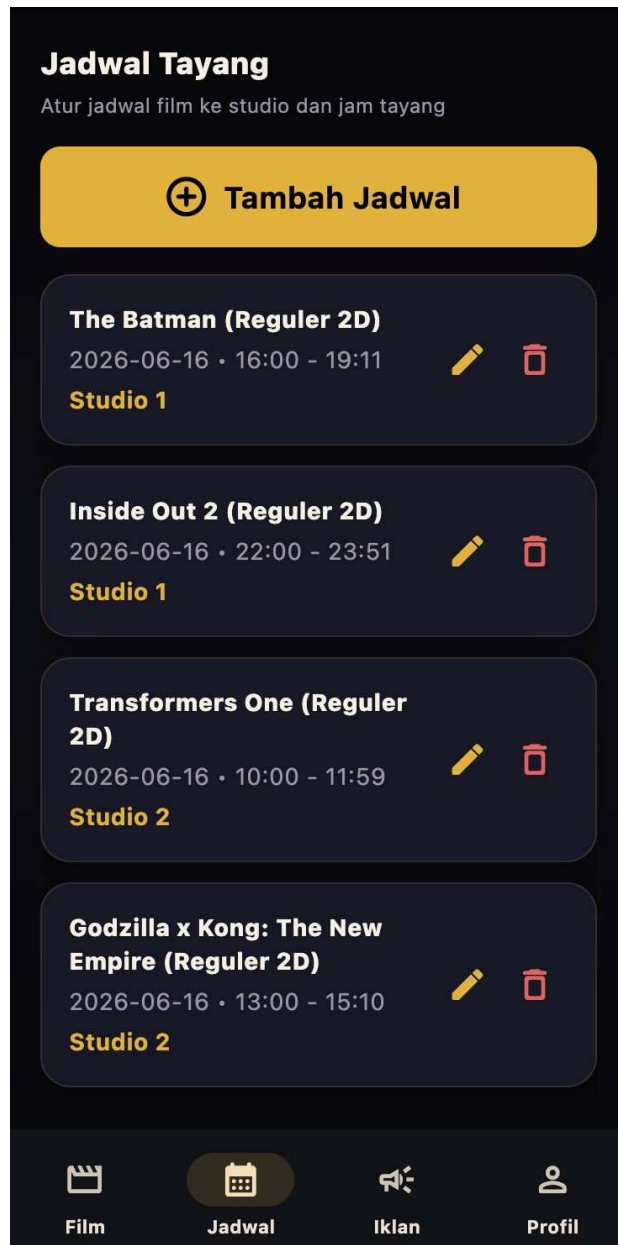
Gambar 42: Halaman profil customer



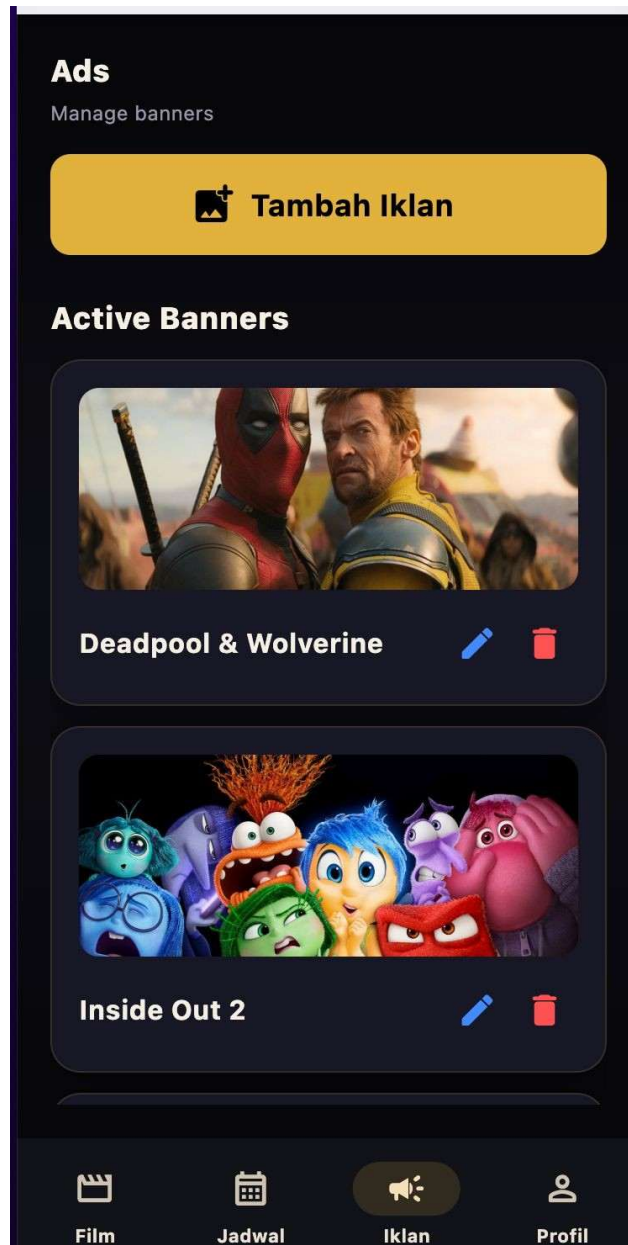
Gambar 43: Halaman profil admin



Gambar 44: Halaman manajemen movie admin



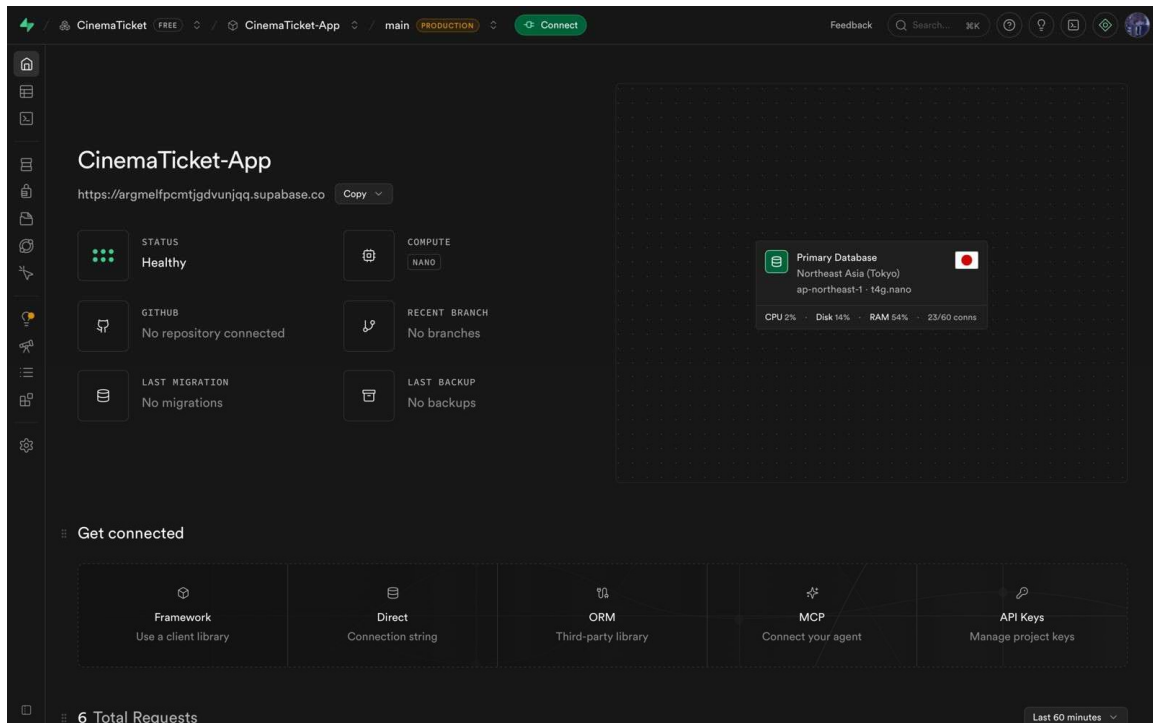
Gambar 45: Halaman manajemen schedule admin



Gambar 46: Halaman manajemen iklan

8.4 Lampiran D Manual Instalasi Sistem

Manual instalasi sistem meliputi persiapan Java, konfigurasi basis data, pengaturan koneksi aplikasi, menjalankan backend, menjalankan frontend, dan melakukan pengujian fitur utama. Dokumentasi berikut menunjukkan bukti konfigurasi basis data Supabase, backend Spring Boot, dan frontend Flutter yang digunakan pada proyek *Cinema Ticket*.



Gambar 47: Konfigurasi dan status koneksi Supabase

```

[INFO] --- compiler3.11.0:testCompile (default-testCompile) @ cinema ---
[INFO] No sources to compile
[INFO] --- spring-boot3.2.4:run (default-cli) < test-compile @ cinema > ---
[INFO] --- spring-boot3.2.4:run (default-cli) @ cinema ---
[INFO] Attaching agents: []

Spring Boot
(3.2.4)

2026-06-16T20:56:32.334+07:00 INFO 33377 [cinema] [main] cinema.Main : Starting Main using Java 17.0.19 with PID 33377 (/Users/kia/Documents/CinemaTicket3/cinema-ticket/backend)
2026-06-16T20:56:32.336+07:00 INFO 33377 [cinema] [main] cinema.Main : No active profile set, falling back to 1 default profile: "default"
2026-06-16T20:56:32.593+07:00 INFO 33377 [cinema] [main] s.s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-06-16T20:56:32.620+07:00 INFO 33377 [cinema] [main] s.s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 24 ms. Found 8 JPA repository interfaces.
2026-06-16T20:56:32.833+07:00 INFO 33377 [cinema] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8082 (http)
2026-06-16T20:56:32.848+07:00 INFO 33377 [cinema] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-06-16T20:56:32.848+07:00 INFO 33377 [cinema] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.19]
2026-06-16T20:56:32.878+07:00 INFO 33377 [cinema] [main] o.s.c.c.c.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-06-16T20:56:32.871+07:00 INFO 33377 [cinema] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 519 ms
2026-06-16T20:56:32.946+07:00 INFO 33377 [cinema] [main] o.hibernate.jpa.internal.util.LogHelper : HHH000284: Processing PersistenceUnitInfo [name: default]
2026-06-16T20:56:32.973+07:00 INFO 33377 [cinema] [main] org.hibernate.Version : HHH000422: Hibernate ORM core version 6.4.4.Final
2026-06-16T20:56:32.989+07:00 INFO 33377 [cinema] [main] o.h.c.internal.RegionFactoryInitiator : HHH00026: Second-level cache disabled
2026-06-16T20:56:33.092+07:00 INFO 33377 [cinema] [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-06-16T20:56:33.183+07:00 INFO 33377 [cinema] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-06-16T20:56:34.631+07:00 INFO 33377 [cinema] [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@8365cef67
2026-06-16T20:56:34.632+07:00 INFO 33377 [cinema] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-06-16T20:56:34.776+07:00 WARN 33377 [cinema] [main] org.hibernate.orm.deprecation : HHH9000025: PostgreSQLDialect does not need to be specified explicitly using 'hibernate.to.dialect' (remove the property setting and it will be selected by default)
2026-06-16T20:56:35.588+07:00 INFO 33377 [cinema] [main] o.h.e.s.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-06-16T20:56:35.183+07:00 INFO 33377 [cinema] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-06-16T20:56:39.252+07:00 INFO 33377 [cinema] [main] o.s.d.j.r.query.QueryEnhancerFactory : Hibernate is in classpath; If applicable, HQL parser will be used.

Hibernate:
select
  ui_0.user_id,
  ui_0.user_type,
  ui_0.email,
  ui_0.name,
  ui_0.password,
  ui_0.balance
from
  users ui_0
where
  ui_0.email=?

2026-06-16T20:56:39.678+07:00 WARN 33377 [cinema] [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-06-16T20:56:39.848+07:00 INFO 33377 [cinema] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8082 (http) with context path ''
2026-06-16T20:56:39.844+07:00 INFO 33377 [cinema] [main] cinema.Main : Started Main in 7.657 seconds (process running for 7.887)
2026-06-16T21:09:43.761+07:00 INFO 33377 [cinema] [nio-8082-exec-1] o.s.c.c.c.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-06-16T21:09:43.772+07:00 INFO 33377 [cinema] [nio-8082-exec-1] o.s.w.s.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-06-16T21:09:43.827+07:00 INFO 33377 [cinema] [nio-8082-exec-1] o.s.w.s.servlet.DispatcherServlet : Completed initialization in 54 ms
```

Gambar 48: Backend Spring Boot berjalan

```

Starting application from main method in: org.dartlang-app/web_entrypoint.dart.
The Dart compiler exited unexpectedly.
(base) KiaMacBook-Pro-kia:frontend % flutter run
Connected devices:
macOS (desktop) • macos • darwin-arm64 • macOS 26.0.1 25A362 darwin-arm64
Chrome (web) • chrome • web-javascript • Google Chrome 149.0.7827.114

Checking for wireless devices...

[1]: macOS (macos)
[2]: Chrome (chrome)
Please choose one (or "q" to quit): 1
Resolving dependencies...
Downloading packages...
cli_util 0.4.2 (0.5.1 available)
file_picker 8.3.7 (11.0.2 available)
matcher 0.12.19 (0.12.20 available)
meta 1.17.0 (1.18.3 available)
qr 3.0.2 (4.0.0 available)
test_api 0.7.18 (0.7.12 available)
vector_math 2.2.0 (2.4.0 available)
win32 5.15.0 (6.3.0 available)
Got dependencies!
8 packages have newer versions incompatible with dependency constraints.
Try 'flutter pub outdated' for more information.
Launching lib/main.dart on macOS in debug mode...
Running pod install... 2,660ms
warning: Run script build phase 'Run Script' will be run during every build because it does not specify any outputs. To address this issue, either add output dependencies to the script phase, or configure it to run in every build by unchecking "Based on dependency analysis" in the script phase. (in target 'Flutter Assemble' from project 'Runner')
Building macOS application...
✓ Built build/macos/Build/Products/Debug/TITIXX PREMIERE.app
2026-06-16 21:41:18.408 TITIXX PREMIERE[48124:562731] Running with merged UI and platform thread. Experimental.
Syncing files to device macOS... 540ms

Flutter run key commands.
r Hot reload. 🚀
R Hot restart.
h List all available interactive commands.
d Detach (terminate "flutter run" but leave application running).
c Clear the screen
q Quit (terminate the application on the device).

A Dart VM Service on macOS is available at: http://127.0.0.1:62367/w19SxIPmkRE=/
The Flutter DevTools debugger and profiler on macOS is available at: http://127.0.0.1:62367/w19SxIPmkRE=/devtools/?uri=ws://127.0.0.1:62367/w19SxIPmkRE=ws
```

Gambar 49: Frontend Flutter berjalan